Deep Metric Learning for Underwater Place Recognition Using Side-Scan Sonar Imagery (Stage 2ème année)

Killian Kerlau

July-August 2026

Contents

1 Introduction

With the recent development of Autonomous Underwater Vehicles (AUVs), geolocalization has become a crucial challenge. Electromagnetic waves attenuate very rapidly underwater, making GPS technology unusable. Currently, an AUV's position is estimated using its inertial navigation system combined with its initial immersion point. However, after several hours of navigation, the dead-reckoning system accumulates multiple meters of drift, making periodic recalibration indispensable.

A major bottleneck in training deep neural networks for underwater robotics is the scarcity of available public sonar databases, as most acoustic surveys remain private due to confidentiality and defense applications. In this work, we use a real-world side-scan sonar dataset collected in Provincetown Bay. We evaluate a Siamese architecture paired with a three-CNN backbone to investigate whether a deep learning model can recognize specific seafloor areas across different viewpoints without prior position knowledge. Because both terrestrial and underwater terrains exhibit distinctive natural features over sufficiently large areas, we explore whether neural networks can effectively leverage these structural characteristics for place recognition and spatial matching.

2 Database Creation

2.1 Raw Data

Data were collected in Provincetown Bay using acoustic and navigation sensors deployed on the vessel *Maradin 6205*. The raw dataset consists of time-stamped, independent measurements acquired by the following systems:

- **GPS Position:** Geographic coordinates (latitude and longitude, WGS-84) and projected coordinates (easting and northing, UTM Zone 19N) recorded by a Trimble R10 receiver. The GNSS antenna is located at an offset of $X = 0.0$ m, $Y = 0.33$ m, and $Z = -2.836$ m relative to the vessel reference frame (see Figure 1).
- **Sound Velocity:** Real-time sound velocity measurements provided directly by the EdgeTech 6205 integrated probe for local acoustic propagation corrections.
- **Attitude:** Roll, pitch, and heave time-series measured by a Teledyne DMS-05 motion reference unit.
- **Heading:** Supplementary true heading data provided by a Hemisphere VS100 GNSS compass.
- **Sonar Data:** Co-registered bathymetry and dual-frequency side-scan sonar backscatter imagery (low-frequency at 230 kHz and high-frequency at 550 kHz) acquired by an EdgeTech 6205 phase-measuring bathymetric sonar.

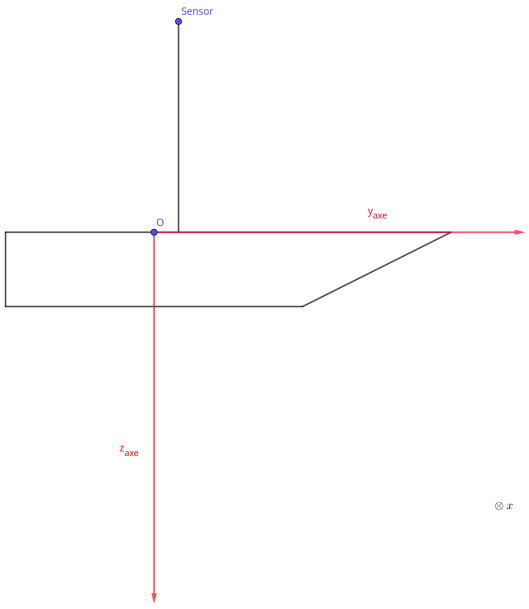


Figure 1: Schematic diagram of the vessel layout and specific sensor lever arms relative to the vessel reference frame.

To construct the patch database, the raw time-series are preprocessed. Parameters including roll, pitch, heave, heading, and geographic coordinates are encapsulated into a dedicated `Measurement` structure. Values at arbitrary sonar ping timestamps are synchronized via linear interpolation. For angular parameters, values are first mapped onto the Cartesian unit circle $(\cos\theta, \sin\theta)$ prior to interpolation to eliminate phase-wrapping discontinuities $(-\pi/\pi)$, before converting back to angles. Finally, local depth profiles are extracted from the phase-difference bathymetric arrays.

2.1.1 Bathymetric Computation

Raw bathymetric arrays from the EdgeTech 6205 provide direction-of-arrival (DOA) angles alongside acoustic quality metrics. First, soundings failing predefined quality thresholds are filtered out to suppress acoustic backscatter noise induced by water-column reverberation, surface reflections, and aeration bubbles.

The measured arrival angle is then corrected for the physical mounting angle of the transducer array and the instantaneous vessel roll:

$$\theta_{\text{estimated}} = \theta_{\text{measured}} - \theta_{\text{mounting}} - \theta_{\text{roll}} \tag{1}$$

where the nominal mounting angles on the port and starboard transducers are $\theta_{\text{port}} = 0.18$ rad and $\theta_{\text{starboard}} = -0.28$ rad, respectively.

Combining local sound velocity and corrected angles allows seafloor profile reconstruction. The reference nadir depth for each ping is defined as the mean depth between the innermost valid starboard sounding and the innermost valid port sounding.

Furthermore, two local topographic descriptors are computed:

- **Flatness:** Evaluated as the first-order slope coefficient from a local linear regression of the seafloor profile.
- **Roughness:** Computed as the standard deviation of the local spatial derivative $(\partial z/\partial x)$ across the cross-track depth profile.

2.2 Georeferencing and Coordinate Transformations

At each sonar ping, the absolute position of the GNSS antenna is recorded. The $(0, 0, 0)$ origin of the vessel reference frame is reconstructed, and the absolute spatial position of the transducer acoustic center is computed by applying successive 3D rotation matrices using the calibrated sensor lever arms:

$$\vec{L}_{\text{GPS}} = \begin{bmatrix} X_0 \\ Y_0 \\ Z_0 \end{bmatrix} = \begin{bmatrix} 0.0 \\ 0.33 \\ -2.836 \end{bmatrix} \quad \text{(in meters)} \tag{2}$$

$$\vec{L}_{\text{Transducer}} = \begin{bmatrix} X_1 \\ Y_1 \\ Z_1 \end{bmatrix} = \begin{bmatrix} 0.0 \\ 0.297 \\ 0.338 \end{bmatrix} \quad \text{(in meters)} \tag{3}$$

The elementary attitude rotation matrices for roll (r), pitch (p), and yaw (y) are formulated as:

$$R_{\text{roll}}(r) = \begin{bmatrix} \cos(r) & 0 & \sin(r) \\ 0 & 1 & 0 \\ -\sin(r) & 0 & \cos(r) \end{bmatrix}, \quad R_{\text{pitch}}(p) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(p) & -\sin(p) \\ 0 & \sin(p) & \cos(p) \end{bmatrix} \tag{4}$$

The yaw rotation matrix is oriented to align directly with the UTM grid (0° at True North, clockwise positive):

$$R_{\text{yaw}}(y) = \begin{bmatrix} \sin(y) & \cos(y) & 0 \\ \cos(y) & -\sin(y) & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{5}$$

The complete orientation matrix R is given by:

$$R = R_{\text{yaw}}(y) \cdot R_{\text{pitch}}(p) \cdot R_{\text{roll}}(r) \tag{6}$$

Accounting for both GPS antenna and transducer lever arms, the absolute georeferenced position of the transducer acoustic center $(E_{\text{sonar}}, N_{\text{sonar}}, Z_{\text{sonar}})$ projected into UTM coordinates is expressed as:

$$\begin{bmatrix} E_{\text{sonar}} \\ N_{\text{sonar}} \\ Z_{\text{sonar}} \end{bmatrix} = \begin{bmatrix} E_{\text{GPS}} \\ N_{\text{GPS}} \\ 0 \end{bmatrix} - R \cdot \vec{L}_{\text{GPS}} + R \cdot \vec{L}_{\text{Transducer}} + \begin{bmatrix} 0 \\ 0 \\ -\text{Heave} \end{bmatrix} \tag{7}$$

Discussion on slant-range projection: In standard real-time processing, a local flat seafloor assumption is adopted. Given the sound speed c and estimated altitude/depth at the current ping, the slant range r and cross-track ground distance x relative to the transducer are calculated as:

$$r = \frac{c \cdot \delta t \cdot n_{\text{index}}}{2}, \quad x = \sqrt{r^2 - \text{depth}^2} \tag{8}$$

While computationally fast, this assumption introduces slight geometric distortions in areas featuring complex bathymetric relief or steep gradients.

2.2.1 Detection Range Index

To quantify the relative geometric position of acoustic features across the sonar swath, a normalized Detection Range Index (DRI $\in [0, 1]$) is defined:

$$\text{DRI} = \frac{|i_{\text{target}} - i_{\text{nadir}}|}{\frac{n_{\text{samples}}}{2}} \tag{9}$$

where i_{target} denotes the cross-track sample index of the target, i_{nadir} is the sample index corresponding to the nadir point, and n_{samples} is the total number of cross-track acoustic samples per ping across both channels. An index of DRI $= 0$ corresponds to nadir, while DRI $= 1$ denotes the outer swath limit.

2.3 Dataset Configurations and Structural Variants

We standardize our extraction matrix resolution to 3000×100 pixels (samples $\times$ pings), corresponding to an approximate seabed physical coverage of 33 m $\times$ 30 m. While isotropic square patches (e.g., 100×100) would simplify rotational data augmentation, downsampling the cross-track resolution requires spatial filtering, which induces significant acoustic feature attenuation and loss of structural detail [1].

Three principal database variants were generated:

1. **Hand-Picked Centered Dataset (HP-Centered):** A benchmark set of distinctive acoustic Regions of Interest (ROIs), including rocky outcrops, shipwreck structures, and recognizable bedform textures. Coordinate windows were centered directly on the calculated UTM coordinates of these keypoints (see Figure 3).

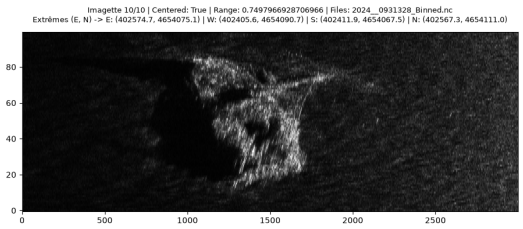
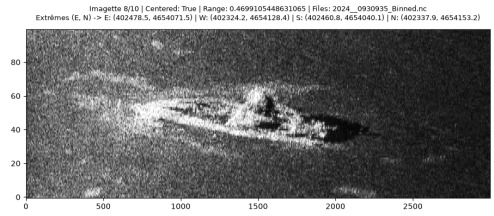
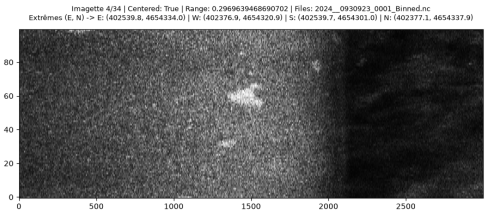


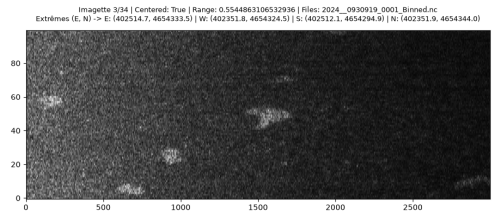
Figure 2: Example of a distinctive rocky structure extracted from Viewpoint 1.



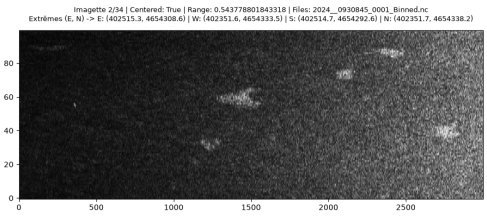
(a) Rock outcrop/wreck (Viewpoint 2)



(b) Seafloor texture (Viewpoint 1)



(c) Seafloor texture (Viewpoint 2)



(d) Seafloor texture (Viewpoint 3)

Figure 3: Representative sonar patch samples across different viewpoints and seabed textures.

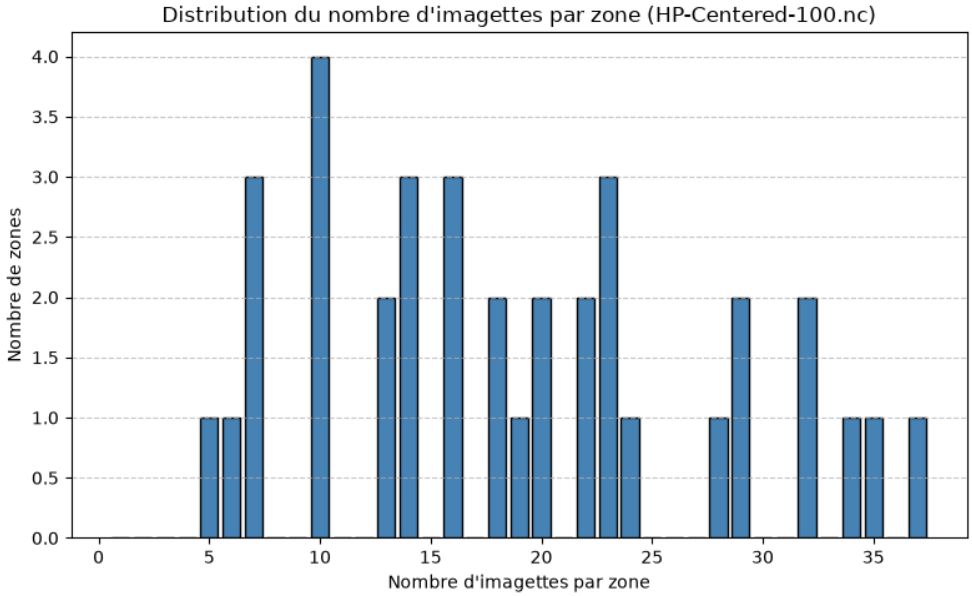


Figure 4: Patch distribution per region of interest in the HP-Centered database.

As shown in Figure 4, class distributions are imbalanced: prominent structural targets (e.g., shipwrecks) have higher multi-pass coverage than subtle sedimentary textures.

2. **Full Regular Grid Dataset (Grid-All):** A continuous geographic partitioning of the surveyed area into uniform 30 m×30 m cells along the vessel trajectory, retaining all patches regardless of morphological salience.

3. **Grid-Aligned Interest Points (Grid-ROI):** Patches extracted using the regular 30 m × 30 m grid framework that intersect identified ROIs, preserving natural uncentered spatial offsets.

The structural properties of the generated datasets are summarized in Table 2.

Table 1: Summary of the generated sonar patch database variants.

Database Variant	Extraction Strategy	Matrix Size	Approx. Extent
HP-Centered 100	ROI-Centered	3000×100	33 m × 30 m
Grid-All 100	Continuous Grid	3000×100	33 m × 30 m
Grid-All-sf 100	Single-file Continuous Grid	3000×100	33 m × 30 m
Grid-All-100-sf-wn	Single-file Grid, Nadir Filtered	3000×100	33 m × 30 m
Grid-All-100-sf-wn-a	Single-file Grid, Augmented	3000×100	33 m × 30 m
Grid-ROI 100	ROI-Filtered Grid	3000×100	33 m × 30 m

2.3.1 Single-File Data Extraction Strategy (Grid-All-sf-100)

In initial iterations, concatenating pings across distinct acquisition files created spatial discontinuity artifacts. Acquisition splits occur when survey parameters are adjusted or when recording files reach size thresholds. Combining pings across non-contiguous survey lines introduced spatial misalignments of several meters within individual patches. The `Grid-All-sf-100` variant resolves this by restricting patch extraction strictly to contiguous pings within identical survey files.

2.3.2 Nadir-Filtered Dataset (Grid-All-100-sf-wn)

To eliminate specular nadir returns and water-column boundary artifacts, patches intersecting the central swath were filtered out. Patches were retained only when satisfying DRI > 0.3. Removing the high-amplitude nadir return improved feature stability across baseline convolutional architectures.

2.3.3 Spatial Data Augmentation (Grid-All-100-sf-wn-a)

To enforce translation invariance, synthetic spatial offsets were applied to extraction windows: along-track translations of ±5 pings and cross-track shifts of ±50 acoustic samples.

2.4 Pair Generation and Split Partitioning

To formulate the place recognition task, matching (positive) and non-matching (negative) patch pairs are generated. An unconstrained combinatorial generation would create an extreme class imbalance dominated by negative pairs. To maintain class balance, valid matching pairs are first formed from co-located multi-pass acquisitions within each zone, and an equal number of non-matching pairs are drawn via uniform negative sampling across distinct geographic cells.

To prevent spatial data leakage, the surveyed region was strictly partitioned into spatially disjoint geographic blocks for Training (60%), Validation (20%), and Testing (20%) prior to pair compilation.

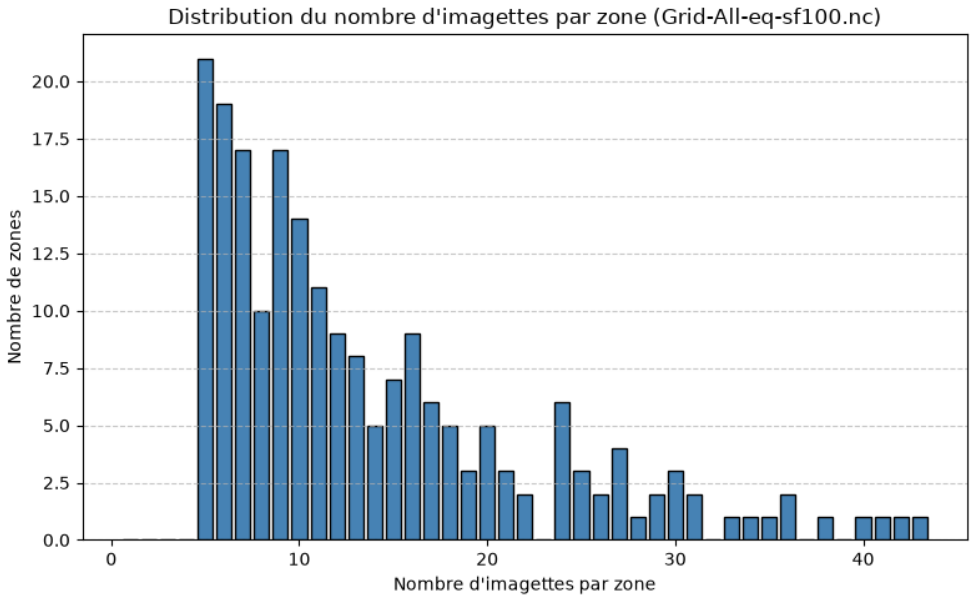


Figure 5: Distribution of patch counts per geographical cell in the continuous Grid-All database.

3 Network Architecture and Learning Methods

3.1 Feature Extraction and Siamese Architectures

Sonar patch registration and matching are formulated as a metric learning problem using a twin-branch Siamese neural network with shared weights. Two primary architectures were developed and evaluated: a baseline three-layer convolutional network (**S-3CNN**) and an enhanced variant incorporating attention mechanisms (**S-3CNN-A**). Both networks are designed modularly to flexibly accommodate different input channel configurations and seamlessly integrate navigation data (notably vessel heading, denoted as H).

3.1.1 Baseline Architecture: 3-Layer Convolutional Siamese Network (S-3CNN / S-H-3CNN)

The baseline architecture relies on an encoder composed of three successive convolutional blocks (**S-3CNN**). Feature extraction is initially performed directly on the acoustic imagery. The 4096 features extracted by the encoder are projected through an intermediate fully connected layer down to a 256-dimensional feature vector.

The network provides flexible options for incorporating auxiliary sensor metadata (such as heading):

- **Vector-Level Fusion (S-H-3CNN):** Scalar metadata variables (n parameters) are concatenated directly with the intermediate 256-dimensional feature vector, yielding a combined $(256 + n)$-dimensional representation.
- **2D Channel-Level Fusion:** Scalar or directional navigation metrics can also be transformed into 2D feature maps (uniform spatial matrices repeating the parameter values) and stacked directly alongside the acoustic image as additional input channels to the CNN.

Finally, a dense output layer projects the combined representation into a 128-dimensional latent embedding space to mitigate the curse of dimensionality, which tends to average out Euclidean distances in overly high-dimensional spaces. The schematic diagram of a single branch of this baseline architecture is shown in Figure 6.

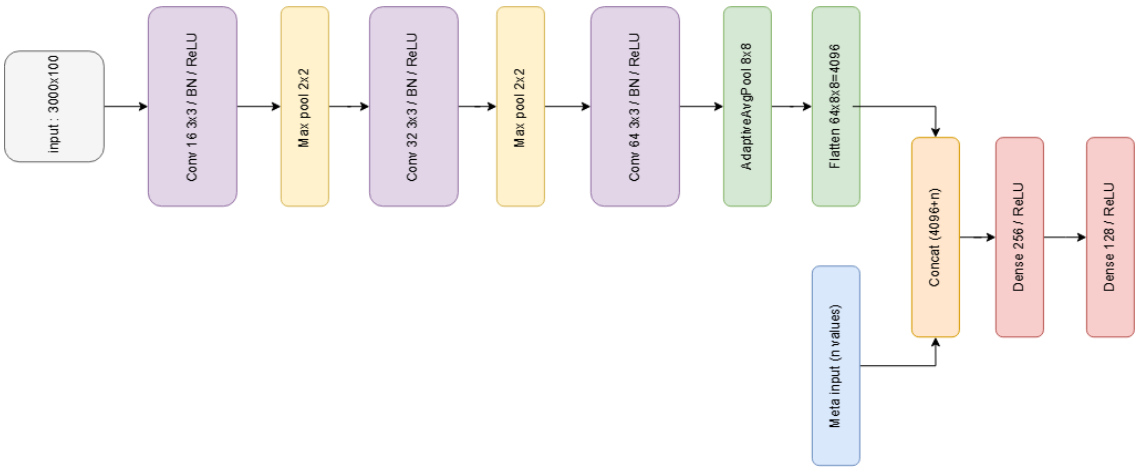


Figure 6: Schematic diagram of one branch of the baseline convolutional Siamese architecture (S-3CNN / S-H-3CNN).

As an additional comparative standard (*baseline*), we also evaluated replacing the 3-layer convolutional block with a **ResNet-50** backbone [2].

3.1.2 Architectural Enhancement: Attention Mechanism Integration (S-3CNN-A)

To suppress acoustic speckle noise and emphasize salient seafloor structural geometries, we propose an architectural extension incorporating a hybrid attention module (**S-3CNN-A** when metadata are integrated). This variant introduces the following mechanisms:

- **Spatial and Channel Attention (CBAM):** Each convolutional stage is augmented with a CBAM block combining channel attention (filtering the most informative feature maps via dual average and max pooling) and spatial attention (focusing on salient acoustic echoes via a 7×7 convolution).

- **Adaptive Pooling:** An `AdaptiveAvgPool2d` layer enforces a fixed 4×4 spatial grid output, ensuring invariant feature dimensionality regardless of input patch dimensions.
- **Projection and ℓ_2-Normalization:** Following optional metadata concatenation, the descriptor passes through a dense layer (256 units with *Dropout* at $p = 0.2$) before being projected to dimension 128 and strictly normalized ($\|v\|_2 = 1$).
- **Cross-Branch Attention (Optional):** A multi-head cross-attention module allows joint pairwise inference to model spatial inter-patch dependencies prior to final metric projection.

3.2 Siamese Network Principle and Objective Functions

3.2.1 Euclidean Distance and Gradient Backpropagation

Let v_a and v_b denote the ℓ_2-normalized feature vectors produced by the twin branches of the Siamese network. Their pairwise Euclidean distance d is defined as:

$$d = \|v_a - v_b\|_2 = \sqrt{\sum_{i=1}^{n}(v_{a,i} - v_{b,i})^2} \tag{10}$$

The derivative of the distance d with respect to an arbitrary feature component $v_{a,i}$ is given by:

$$\frac{\partial d}{\partial v_{a,i}} = \frac{v_{a,i} - v_{b,i}}{d} \quad (d > 0) \tag{11}$$

By applying the chain rule, the gradient of the objective function $\mathcal{L}$ with respect to the feature vector v_a is computed as $\frac{\partial \mathcal{L}}{\partial v_{a,i}} = \frac{\partial \mathcal{L}}{\partial d}\frac{\partial d}{\partial v_{a,i}}$, which is subsequently backpropagated through the convolutional encoder using the Adam optimizer.

3.2.2 Hadsell-Chopra-LeCun Contrastive Loss

For a pair of input patches associated with a binary ground-truth label $Y \in \{0, 1\}$ ($Y = 1$ for genuine/positive pairs, $Y = 0$ for impostor/negative pairs), the contrastive loss function [5] is defined as:

$$\mathcal{L}(v_a, v_b, Y) = Y d^2 + (1 - Y)\max(0, m - d)^2 \tag{12}$$

where $m > 0$ represents the predefined margin constraint for dissimilar pairs. Over a training batch of size B, the empirical batch loss is minimized:

$$\mathcal{L}_{\text{batch}} = \frac{1}{B}\sum_{k=1}^{B}\mathcal{L}_k \tag{13}$$

The partial derivative of $\mathcal{L}$ with respect to the distance d evaluates to:

$$\frac{\partial \mathcal{L}}{\partial d} = \begin{cases} 2d & \text{if } Y = 1 \\ -2(m - d) = 2(d - m) & \text{if } Y = 0 \text{ and } d < m \\ 0 & \text{if } Y = 0 \text{ and } d \geq m \end{cases} \tag{14}$$

When $Y = 1$, the objective forces $d \to 0$. Conversely, when $Y = 0$, the network incurs a penalty only if $d < m$, driving the representations apart until the margin is satisfied.

3.2.3 InfoNCE Loss

As a self-supervised contrastive alternative, the InfoNCE loss [10] is evaluated to leverage positive and multiple negative candidates within the embedding space:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp\left(\text{sim}(q, k_+)/\tau\right)}{\exp\left(\text{sim}(q, k_+)/\tau\right) + \sum_{j=1}^{K}\exp\left(\text{sim}(q, k_j^-)/\tau\right)} \tag{15}$$

where $\text{sim}(u, v) = u^\top v$ denotes the cosine similarity on ℓ_2-normalized embeddings, and τ is a temperature scaling parameter.

3.3 Decision Strategies and Uncertainty Handling

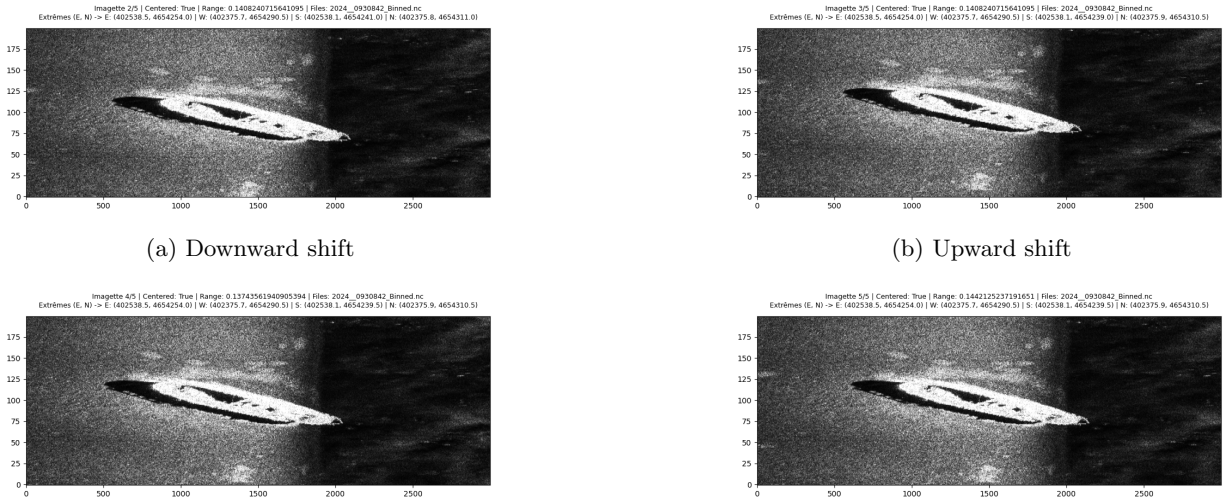
3.3.1 Optimal Hard Thresholding

A standard classification decision is obtained via threshold moving [4]. The optimal distance threshold S is calibrated on the validation set to maximize the discrimination metric (F_1-score and precision).

3.4 Sensitivity Analysis and Model Selection

3.4.1 Spatial Perturbation Protocol

To assess the stability of the feature embeddings against vessel trajectory variations, geometric perturbation tests are conducted across six representative seafloor areas. Four controlled translations (up, down, left, right) are applied to the acoustic patches (300 samples across range, 30 pings along track), as illustrated in Figure 7.



(a) Downward shift

(b) Upward shift

(c) Leftward shift

(d) Rightward shift

Figure 7: Geometric transformations applied to benchmark patch sensitivity.

3.4.2 Water Column Consideration and Dataset Variants

The input acoustic matrices maintain their original 3000×100 dimension (covering approximately $33\,\text{m} \times 33\,\text{m}$ on the seabed). The water column region is intentionally retained to preserve absolute acoustic geometry and avoid truncating near-nadir bottom returns. Table 2 summarizes the sensitivity benchmark configuration.

Table 2: Summary of the sensitivity benchmark dataset configuration.

Dataset Variant	**Matrix Dimensions**	**Perturbation Shifts** (r, θ)
`sensitivity-100`	3000×100	$\pm 300\,\text{px}, \pm 30\,\text{px}$

3.4.3 Training Supervision and Model Checkpointing

Model convergence is monitored across epochs using the mean loss on the training and validation splits. Model weights are saved whenever validation loss achieves a new global minimum, ensuring optimal checkpointing and mitigating overfitting.

4 Experiments

For performance evaluation, a baseline classification threshold was set to 0.5. Note that accuracy metrics could be further optimized through threshold tuning or an adaptive thresholding scheme. Unless explicitly stated otherwise (e.g., when employing InfoNCE loss or varying the batch size), all experiments were conducted using the Hadsell–Chopra–LeCun contrastive loss with a default batch size of 64.

Sur les premieres expériences la tailles du batch était de 16

4.0.1 Experiments 1 and 7

As both models exhibit very similar response characteristics, we primarily report and discuss the results obtained with the attention-augmented architecture (S-3CNN-A).

In this benchmark, we evaluate the baseline model composed of a 3-layer convolutional network. The network takes only the raw sonar patch as input, without incorporating any auxiliary metadata.

It can be observed that the model overfits rapidly (starting from the fourth epoch), which may suggest that the data lack sufficient quality, volume, or geographic diversity.

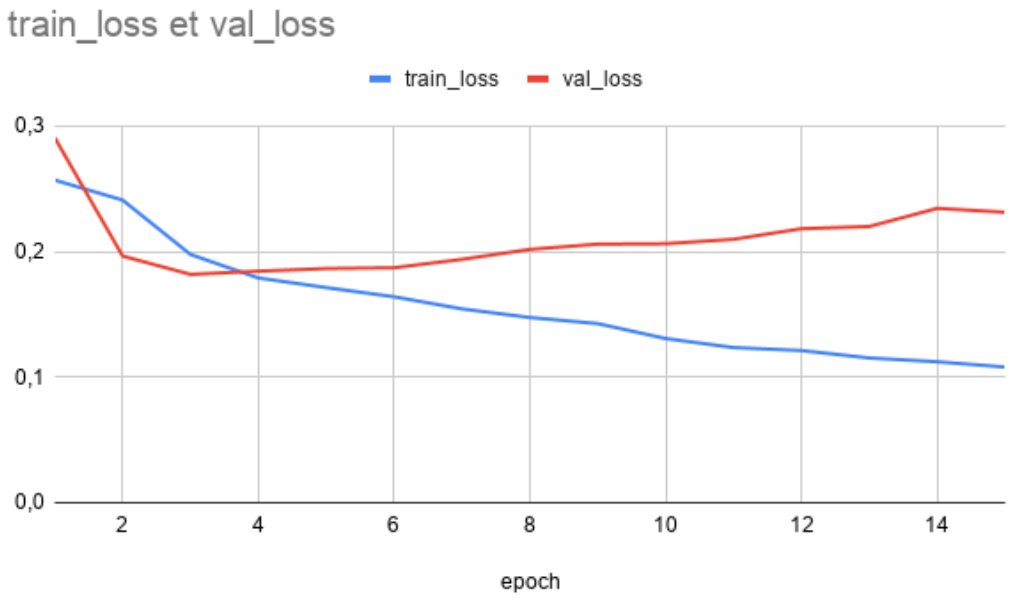


Figure 8: Training and validation loss evolution across epochs for Experiment 1.

As described in Section **??**, model checkpointing is applied to retain only the weights corresponding to the lowest validation loss. The model is optimized using the contrastive loss function proposed by Hadsell et al. [5].

An analysis of the classification behavior on the test split reveals that the network successfully detects the presence of salient acoustic objects within a patch, yet struggles to identify the specific geographic location accurately.

Table 3: Synthèse expériences

n°	Model	Epochs	Dataset	Split	Accuracy	TP	TN	FP	FN
1	S3CNN	15	HP-Centered-100	Train (60%)	70.9%	2843	1487	1566	210
				Eval (20%)	73.1%	2490	1368	1272	150
				Test (20%)	62.8%	1535	628	1094	187
2	S3CNN	15	Grid-All-sf100	Train (60%)	51.15%	35002	1311	34186	495
				Eval (20%)	51.19%	9557	479	9323	245
				Test (20%)	50.96%	11955	469	11722	236
3	S3CNN	15	Grid-All-sf100-wn	Train (60%)	51.72%	18045	1109	17408	472
				Eval (20%)	51.72%	4829	389	4655	215
				Test (20%)	51.63%	6080	398	5876	194
4	S3CNN	5	Grid-All-sf100-wn-a	Train (60%)	63.6%	384859	203008	259106	77255
				Eval (20%)	61.0%	98084	57511	70131	29558
				Test (20%)	57.2%	106940	53674	86690	33424
5	S3CNN-H	15	Grid-All-sf100-wn	Train (60%)	54.94%	15120	5228	13289	3397
				Eval (20%)	56.16%	3924	1741	3303	1120
				Test (20%)	56.30%	5011	2054	4220	1263
6	S3CNN-O-H	15	Grid-All-sf100-wn	Train (60%)	55.79%	15347	5313	13204	3170
				Eval (20%)	56.72%	4015	1707	3337	1029
				Test (20%)	56.94%	5029	2116	4158	1245
7	S3CNN-A	15	HP-Centered-100	Train (60%)	72.9%	2970	1478	1575	83
				Eval (20%)	72.3%	2457	1358	1282	183
				Test (20%)	62.7%	1523	637	1085	199
8	S3CNN-A	15	Grid-All-sf100	Train (60%)	62.7%	32274	12243	23254	3223
				Eval (20%)	51.5%	7346	2752	7050	2456
				Test (20%)	51.8%	9346	3271	8920	2845
9	S3CNN-A	15	Grid-All-sf100-wn	Train (60%)	62.5%	16933	6212	12305	1584
				Eval (20%)	52.8%	3761	1567	3477	1283
				Test (20%)	52.9%	4759	1877	4397	1515
10	S3CNN-A	5	Grid-All-sf100-wn-a	Train (60%)	73.1%	394287	280956	181158	67827
				Eval (20%)	57.9%	73278	74616	53026	54364
				Test (20%)	57.2%	82624	77827	62537	57740
14	S3CNN - InfoNCE	5	Grid-All-sf100-wn	Train (60%)	%				
				Eval (20%)	%				
				Test (20%)	%				
15	ResNet-50	5	Grid-All-sf100-wn	Train (60%)	65.83%	15865	8516	10001	2652
				Eval (20%)	57.08%	3646	2112	2932	1398
				Test (20%)	57.77%	4630	2619	3655	1644
16	ResNet-50	5	HP-Centered-100	Train	72.87%	1261	638	665	42
				Eval	74.74%	909	511	439	41
				Test	63.30%	631	316	432	117

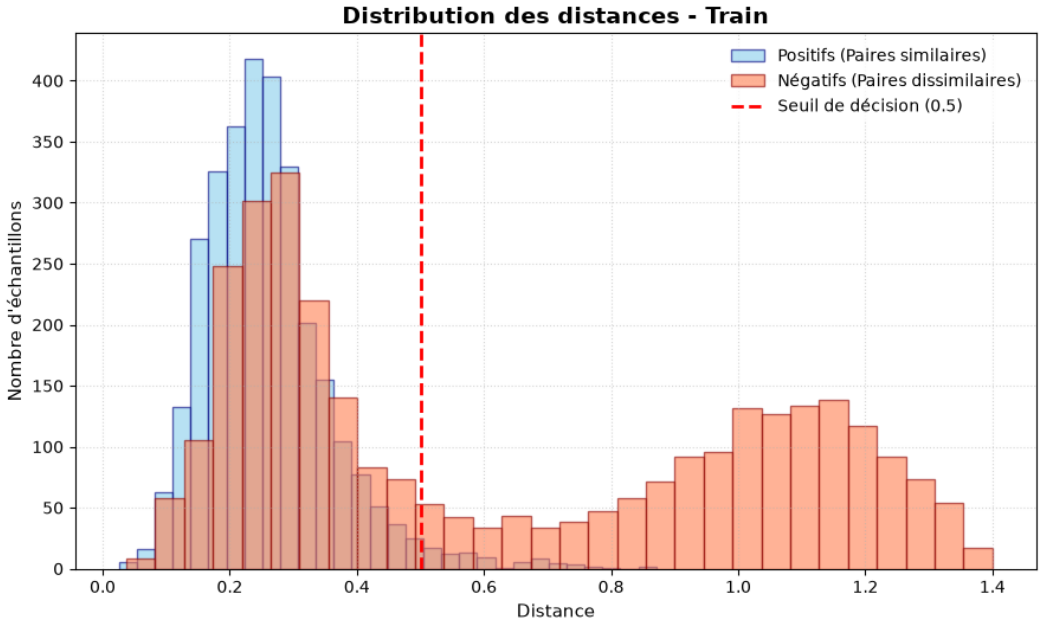


Figure 9: Pairwise Euclidean distance distribution on the training set (Experiment 1).

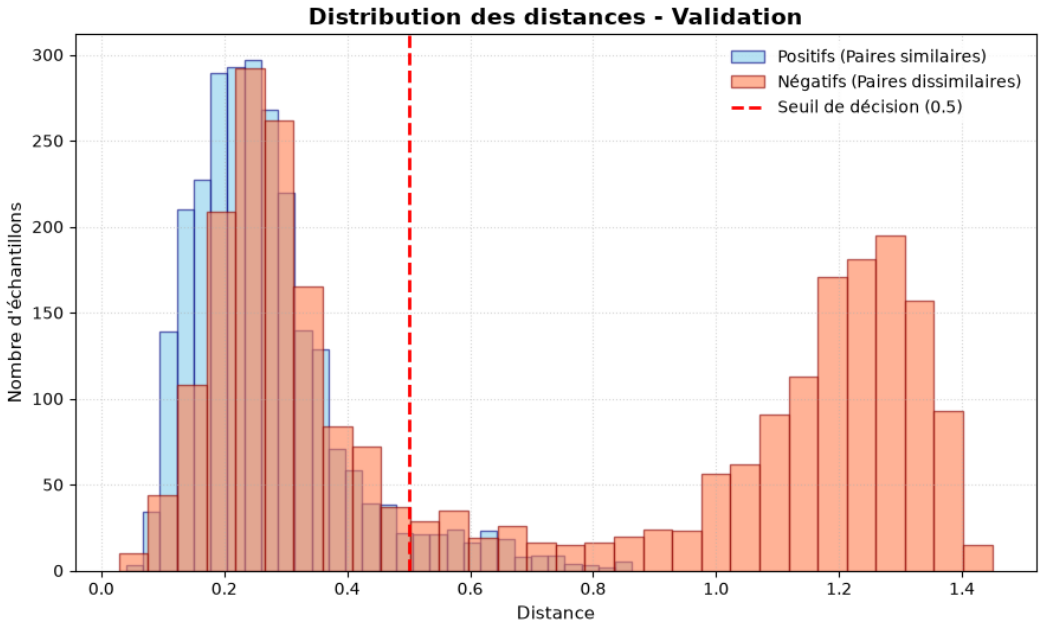


Figure 10: Pairwise Euclidean distance distribution on the validation set (Experiment 1).

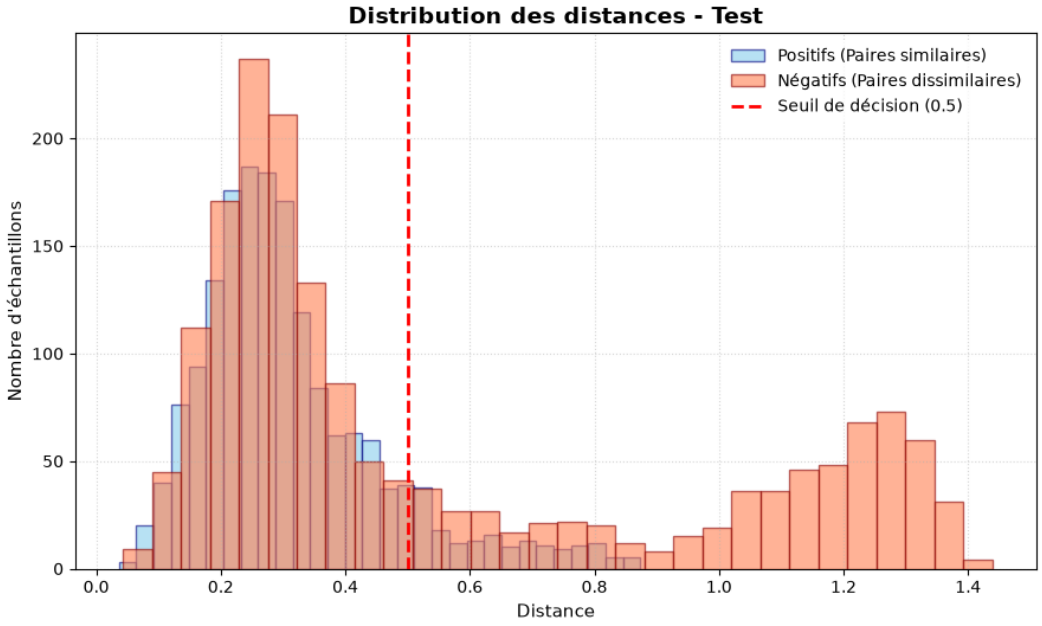


Figure 11: Pairwise Euclidean distance distribution on the test set (Experiment 1).

As illustrated above, while the model correctly identifies a subset of patch pairs, it produces a significant number of false positives on negative pairs.

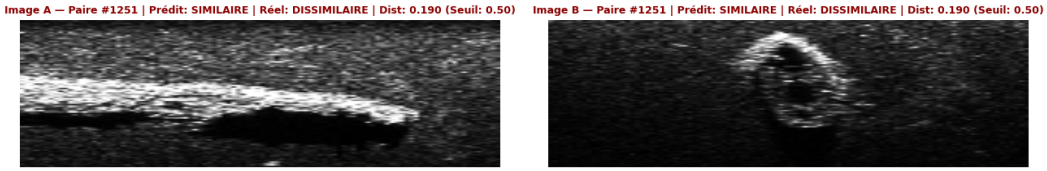


Figure 12: Qualitative comparison on object-to-object dissimilar pairs (Example 1).

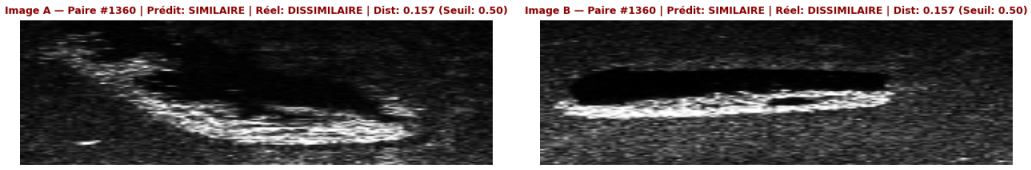


Figure 13: Qualitative comparison on object-to-object dissimilar pairs (Example 2).

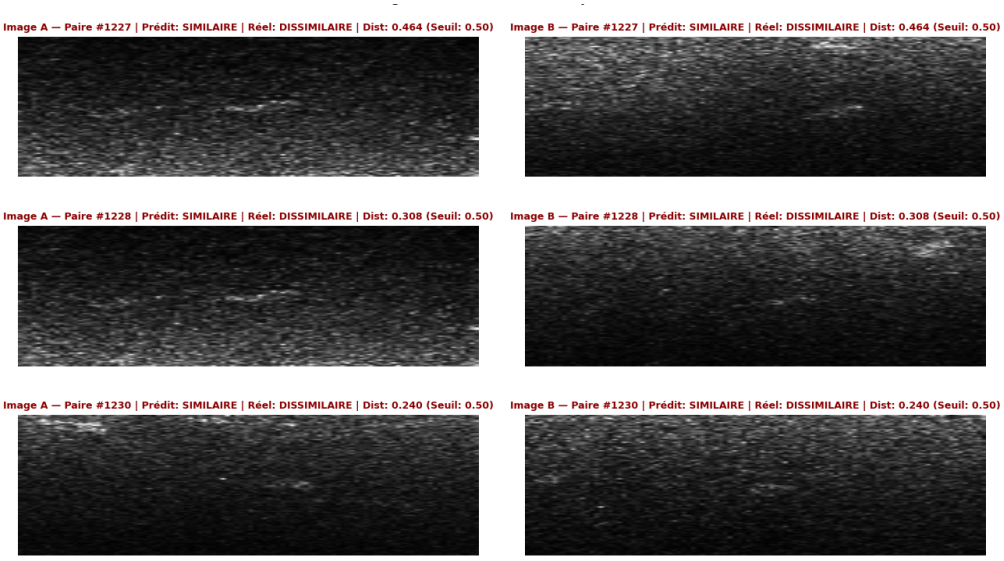


Figure 14: Qualitative comparison on background noise/texture pairs.

Overall, the network exhibits difficulty in discriminating between two distinct structural objects or two homogeneous seafloor textures. Conversely, it reliably separates acoustic targets from uniform background noise. The rare misclassification in this regime generally occur when acoustic speckle or local reverberation artifacts mimic structural features.

4.0.2 Experiment 2

Overfitting becomes apparent starting from the seventh epoch, as the validation loss begins to diverge. Nonetheless, the network manages to develop an initial capability for pattern and shape recognition during the early training phase.

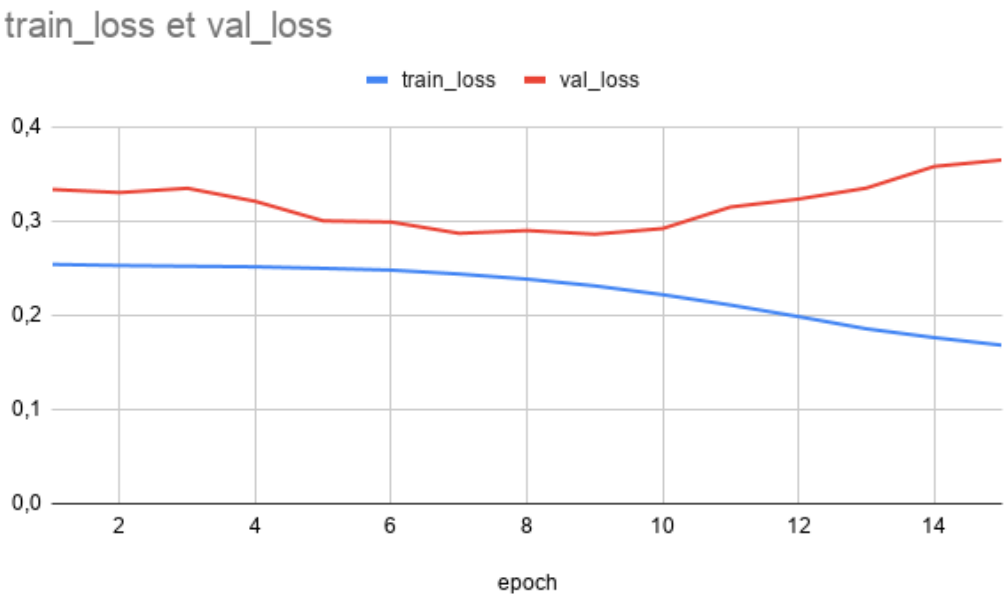


Figure 15: Training and validation loss evolution across epochs for Experiment 2.

Following our model checkpointing strategy, we retain the model weights obtained at the second epoch, which corresponds to the minimal validation loss.

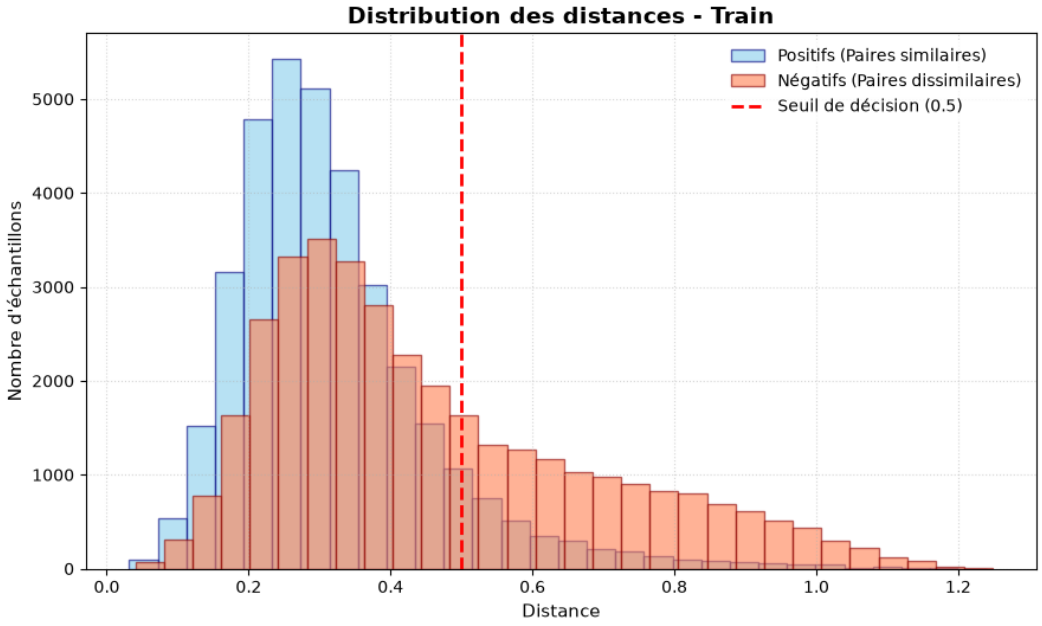


Figure 16: Pairwise Euclidean distance distribution on the training set (Experiment 2).

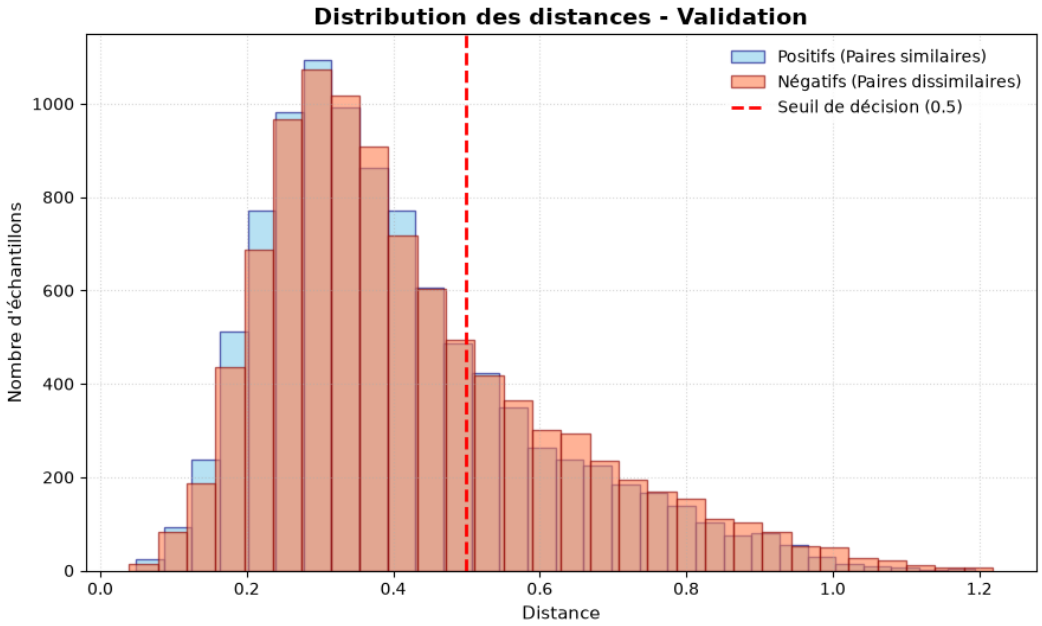


Figure 17: Pairwise Euclidean distance distribution on the validation set (Experiment 2).

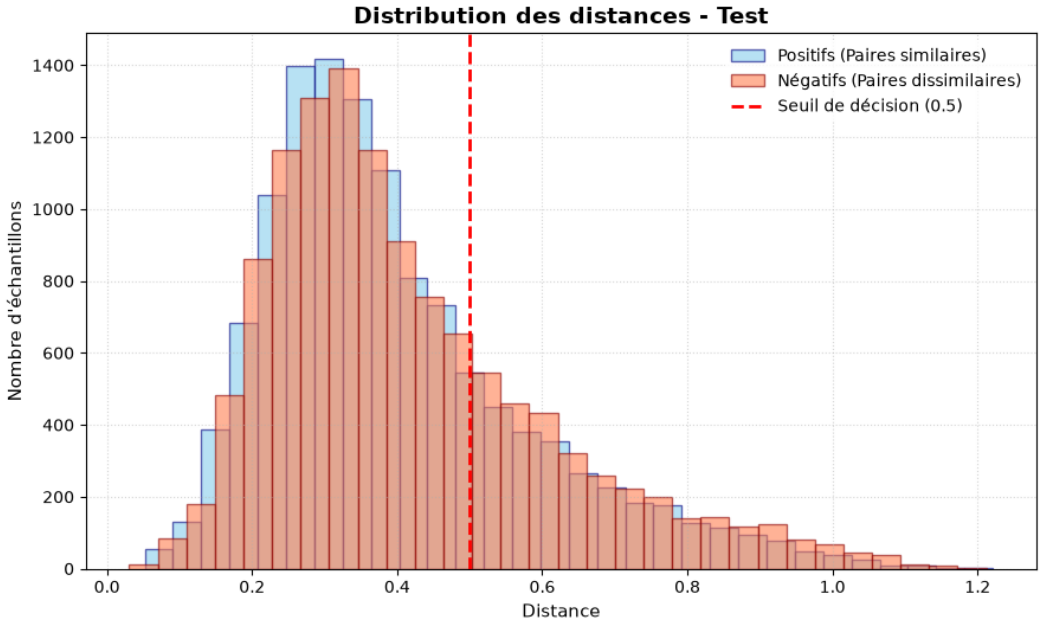


Figure 18: Pairwise Euclidean distance distribution on the test set (Experiment 2).

As evidenced by the overlapping distance distributions on the evaluation and test sets, the network suffers from severe overfitting and remains unable to reliably discriminate between distinct geographic areas.

4.0.3 Experiment 3: Impact of Nadir Region Removal

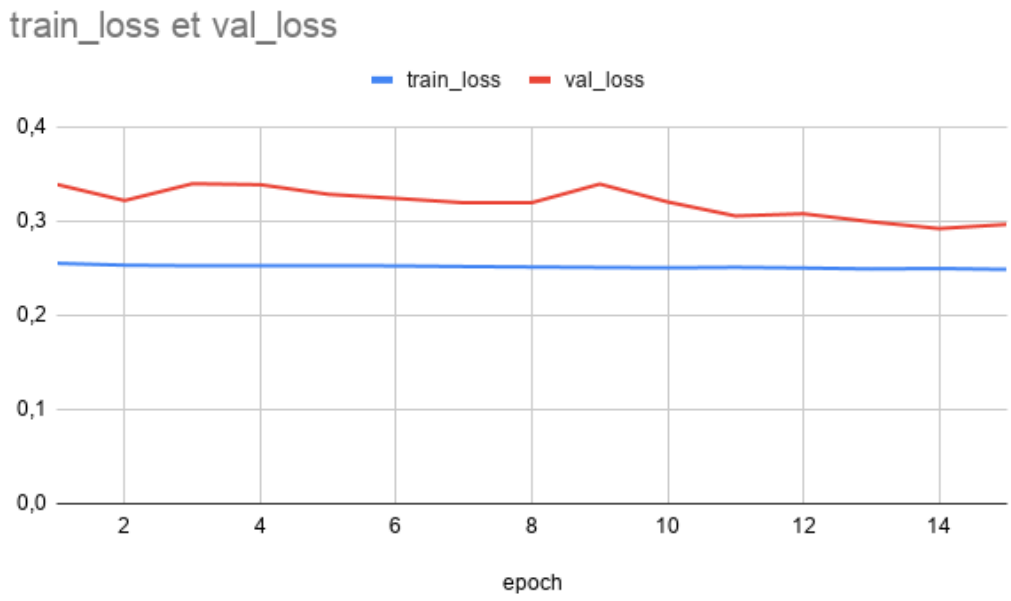


Figure 19: Training and validation loss evolution across epochs for Experiment 3.

As shown in Figure 19, the training dynamics exhibit a concrete improvement: while the training loss remains steady, the validation loss steadily decreases, demonstrating that the model avoids the severe overfitting previously observed in Experiment 2.

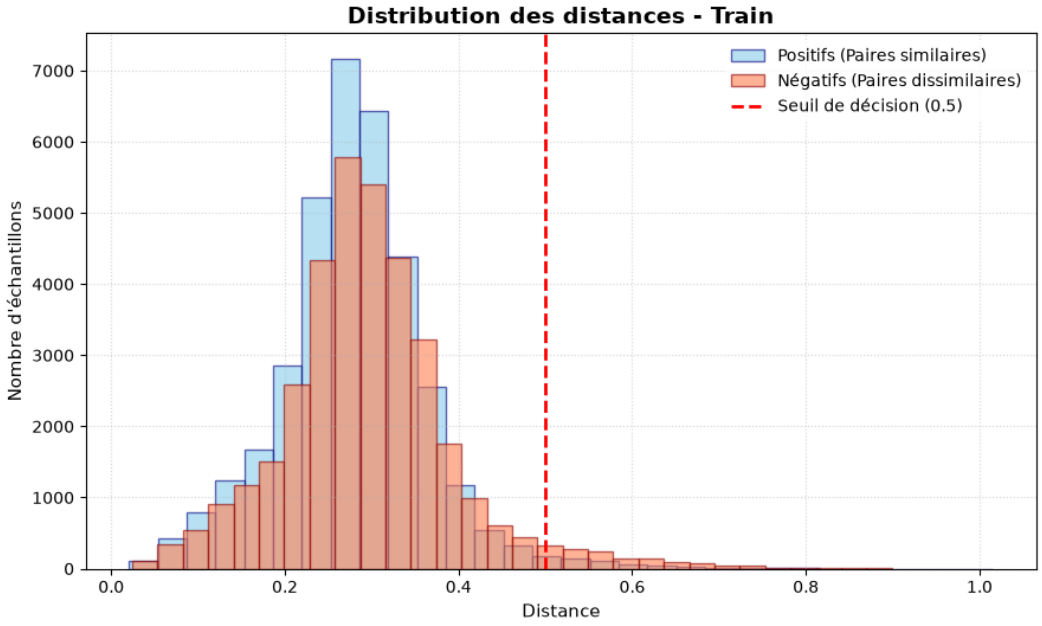


Figure 20: Pairwise Euclidean distance distribution on the training set (Experiment 3).

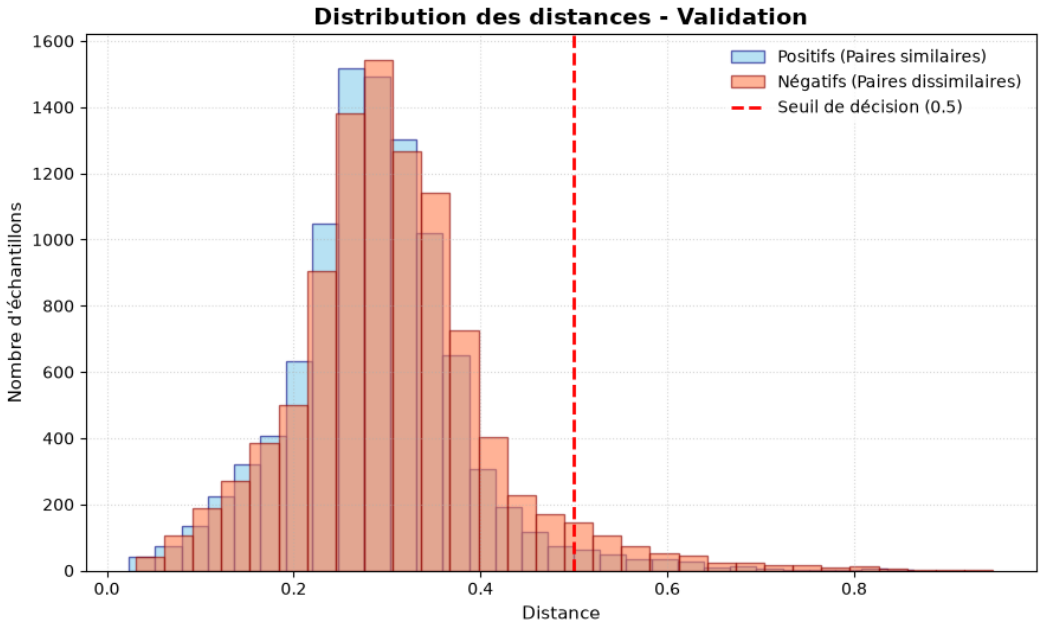


Figure 21: Pairwise Euclidean distance distribution on the validation set (Experiment 3).

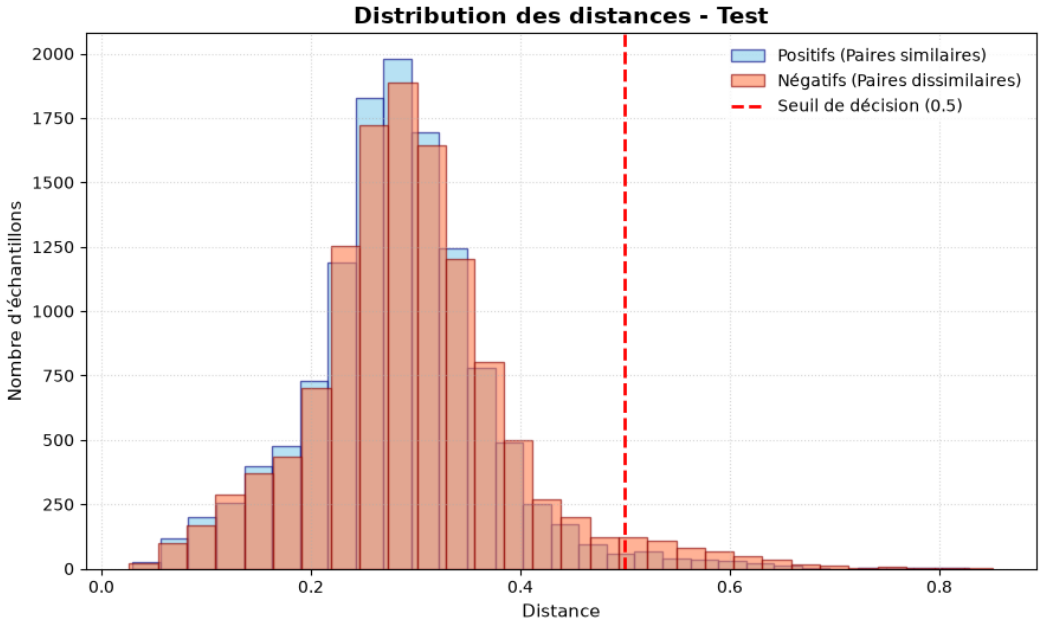


Figure 22: Pairwise Euclidean distance distribution on the test set (Experiment 3).

Filtering out the nadir region slightly improves overall performance and mitigates overfitting; however, the distance distributions remain largely unchanged, indicating that while beneficial, this filtering alone does not represent a fundamental breakthrough.

4.0.4 Experiment 4: Evaluation of the Augmented Dataset (`Grid-All-100-wn-a`)

Augmenting the dataset via slight spatial translations does not appear to genuinely improve the model's global generalization capability. Instead, the network seems prone to early overfitting. As illustrated in Figure 23, the validation loss continuously increases starting from the very first epoch.

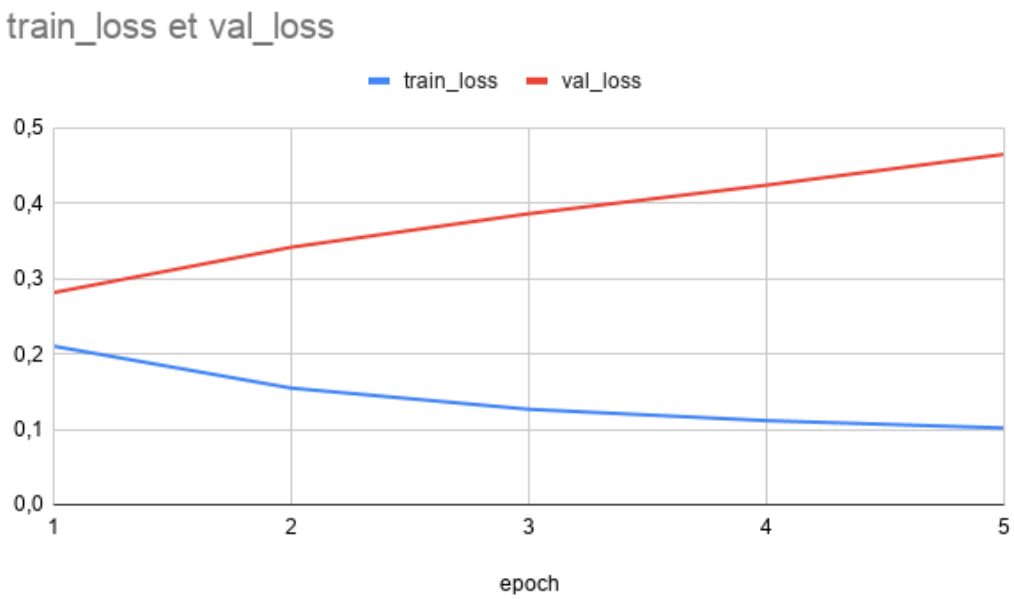


Figure 23: Training and validation loss evolution across 5 epochs for Experiment 4.

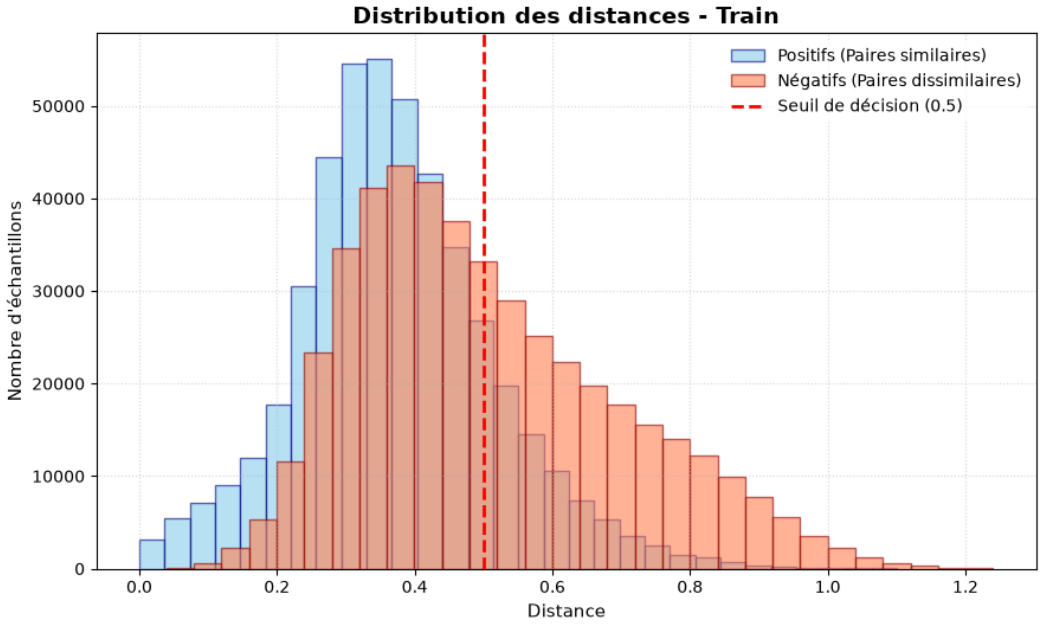


Figure 24: Pairwise Euclidean distance distribution on the training set (Experiment 4).

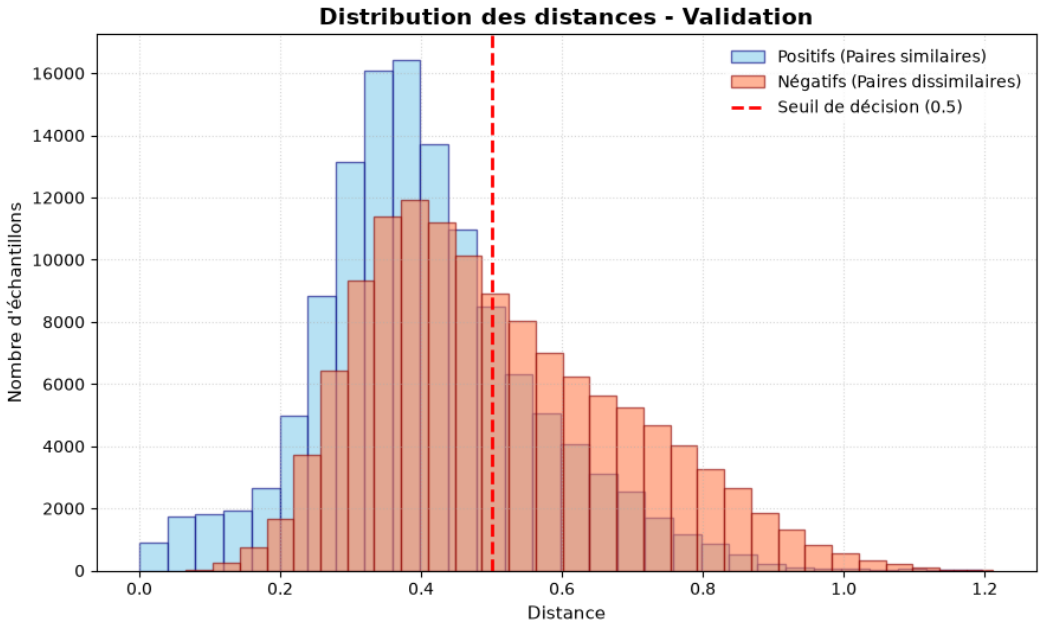


Figure 25: Pairwise Euclidean distance distribution on the validation set (Experiment 4).

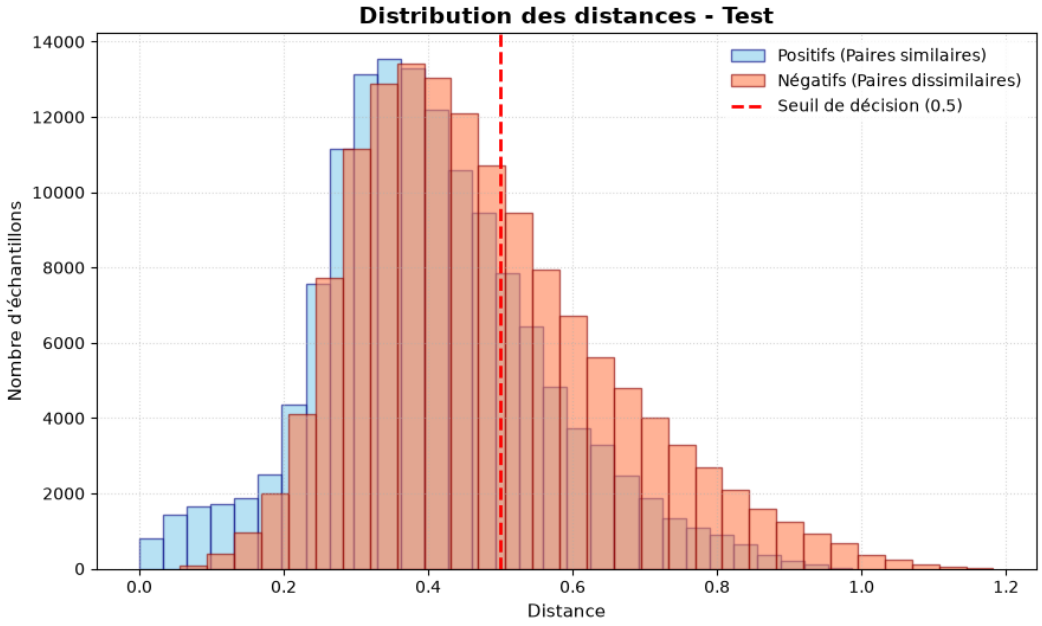


Figure 26: Pairwise Euclidean distance distribution on the test set (Experiment 4).

Although the overall classification accuracy increases by approximately 10% on this augmented dataset, this apparent gain must be interpreted with caution. Because the spatial perturbation strategy quadrupled the number of near-identical patches, this improvement primarily reflects memorization of overlapping samples rather than the acquisition of deeper invariant representations or higher-level semantic features.

4.0.5 Integration of Heading Metadata and Sonar Imagery: Experiments 5 and 6

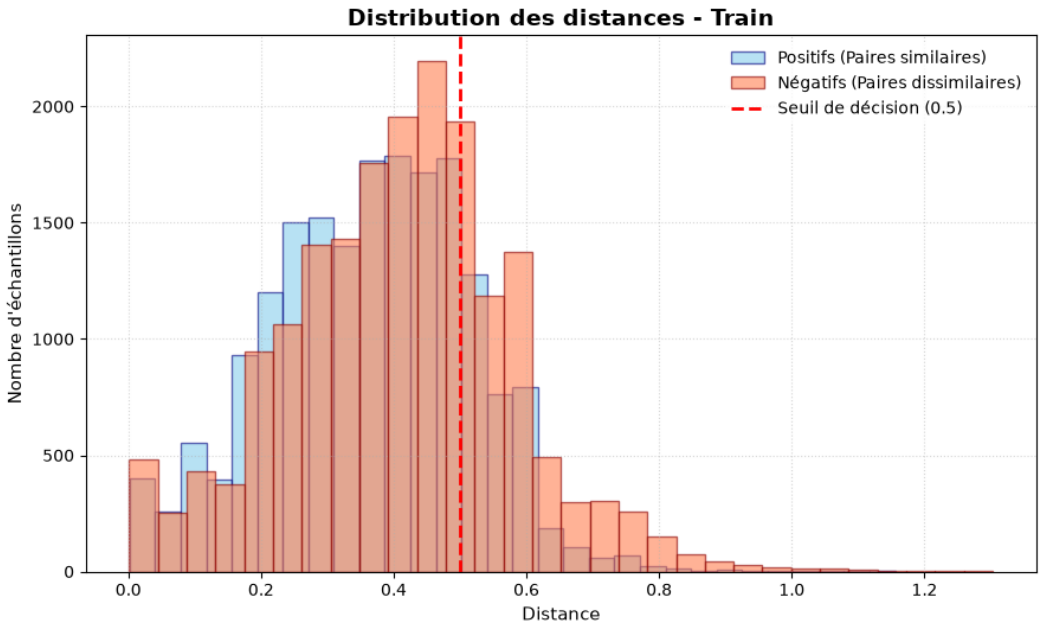


Figure 27: Pairwise Euclidean distance distribution on the training set (Experiment 5: Image + Heading).

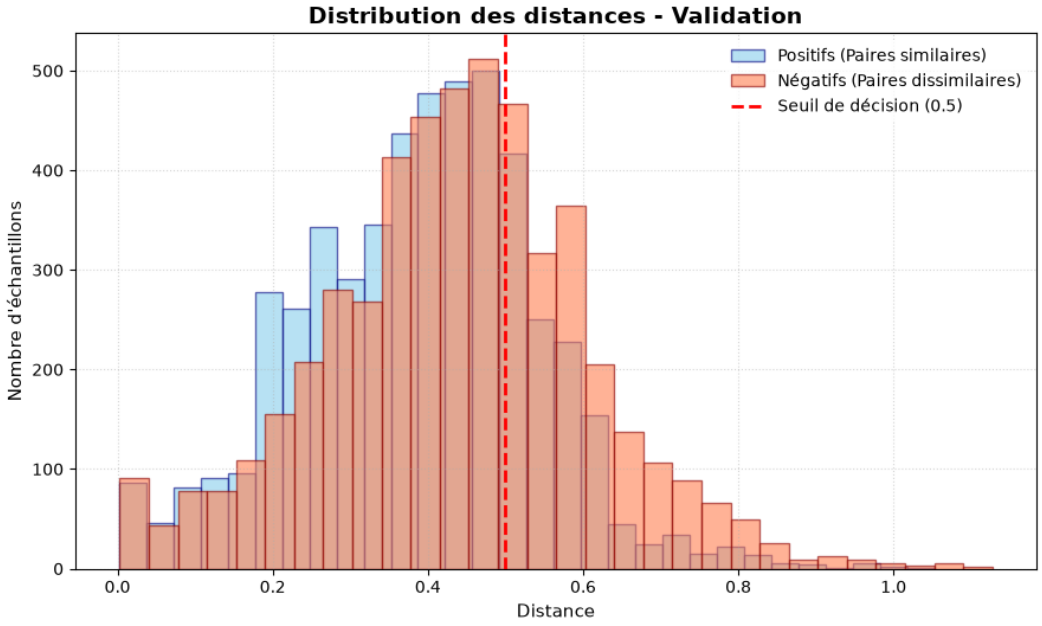


Figure 28: Pairwise Euclidean distance distribution on the validation set (Experiment 5: Image + Heading).

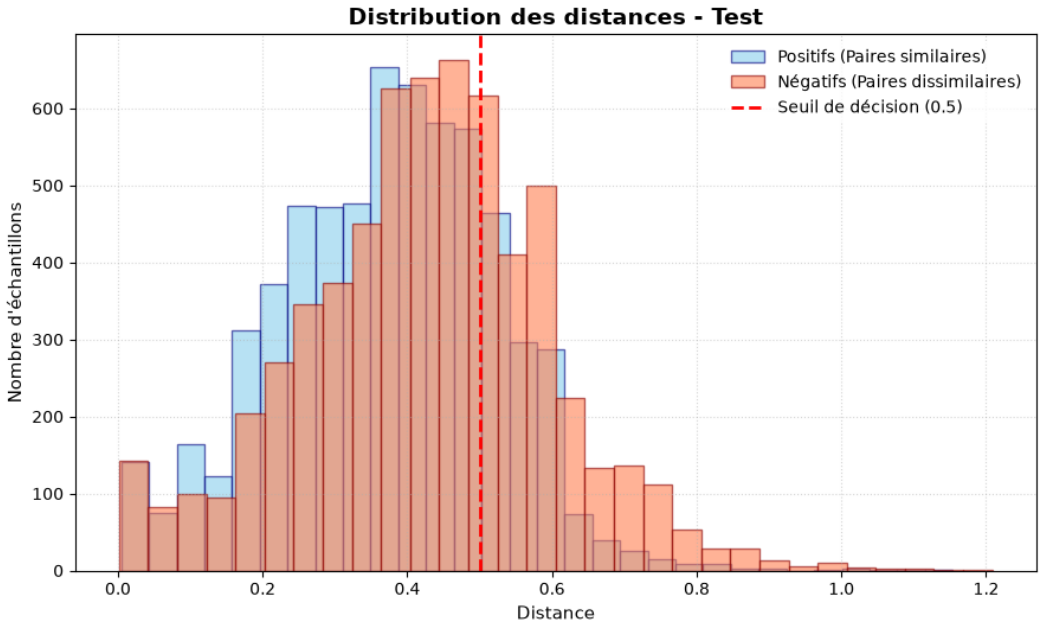


Figure 29: Pairwise Euclidean distance distribution on the test set (Experiment 5: Image + Heading).

Distinct distance distributions emerge between positive and negative pairs, and their profiles remain highly consistent across the training, validation, and test splits. By implementing an adaptive decision threshold, the classification accuracy can be improved to over 70%.

To evaluate whether the network genuinely leverages joint visual-navigational representations or relies predominantly on heading information, Experiment 6 was conducted using exclusively the heading metadata without the acoustic imagery.

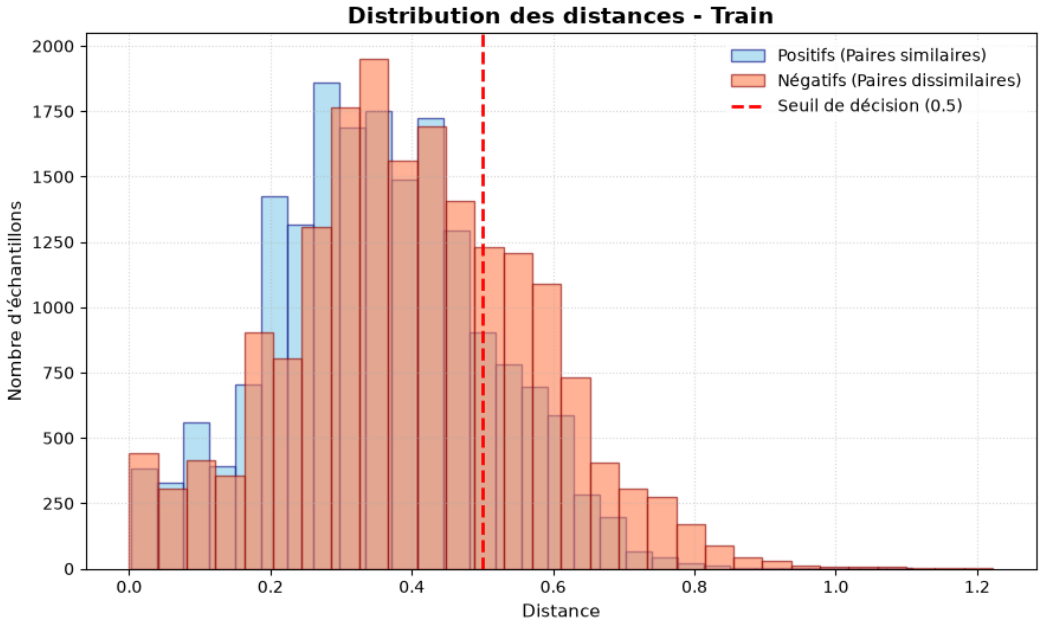


Figure 30: Pairwise Euclidean distance distribution on the training set (Experiment 6: Heading only).

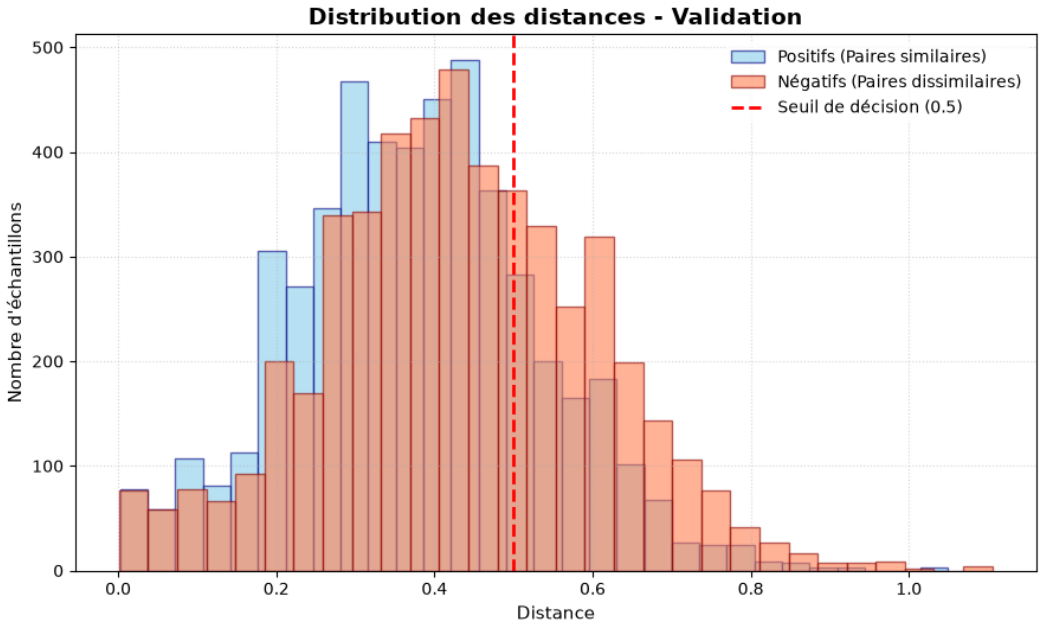


Figure 31: Pairwise Euclidean distance distribution on the validation set (Experiment 6: Heading only).

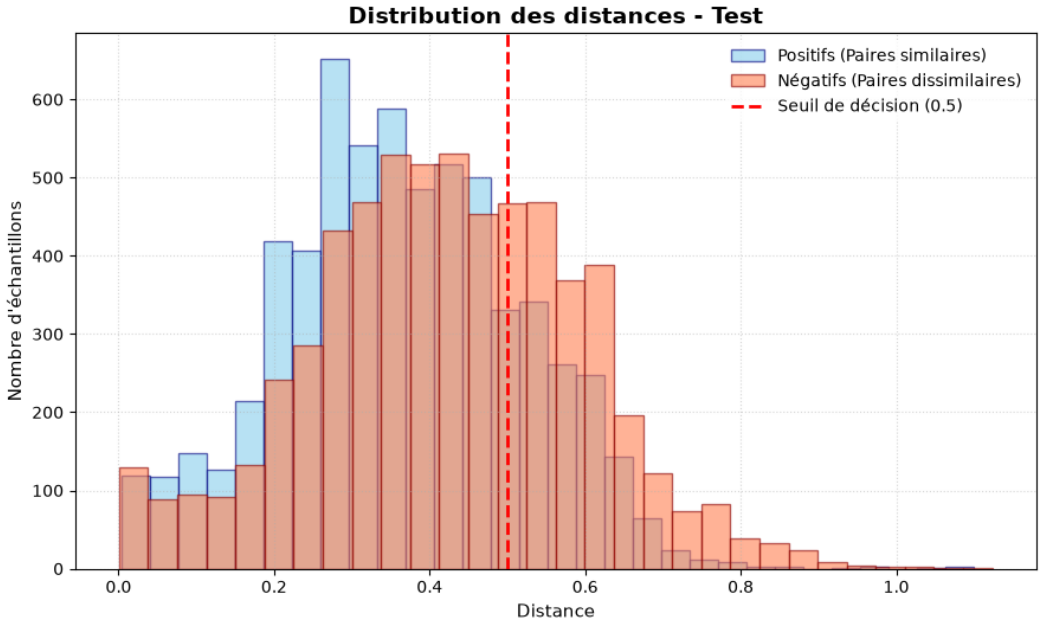


Figure 32: Pairwise Euclidean distance distribution on the test set (Experiment 6: Heading only).

Given the near-identical distance distributions and accuracy levels between Experiments 5 and 6, the model appears to rely almost entirely on heading data rather than extracting discriminating features from the acoustic imagery. This indicates that the network is primarily exploiting spurious correlations between the vessel's trajectory and specific geographic survey zones.

4.0.6 Expérience 14 : Use of InfoNCE Loss

When training with the InfoNCE objective function, the output pairwise distance and similarity distributions for positive and negative pairs remain virtually identical. The network fails to form well-separated clusters in the latent embedding space, indicating that the self-supervised contrastive formulation is unable to extract meaningful invariant acoustic features under the current data and batch constraints.

4.0.7 Experiment 15: Evaluation of the ResNet-50 Backbone

To assess whether deeper hierarchical feature extractors can better capture subtle acoustic bedforms compared to shallow networks, we evaluated a Siamese architecture utilizing a standard **ResNet-50** backbone. Due to GPU memory constraints associated with the increased parameter scale of ResNet-50 on high-resolution sonar matrices (3000×100), the mini-batch size was reduced to $B = 16$.

As shown in Figure 33, the model exhibits rapid convergence during early iterations, reaching its minimum validation loss at epoch 7 before entering an overfitting regime.

Discussion on Performance and Latent Distribution On the test partition, ResNet-50 achieves a baseline accuracy of 57.77%, slightly outperforming both the shallow baseline (`S-3CNN`, 51.63%) and the attention-augmented model (`S-3CNN-A`, 52.9%) on the identical `Grid-All-sf100-wn` benchmark.

The pairwise distance distributions (Figure 34) demonstrate a clearer structural separation between genuine and impostor pairs in the latent space:

- **Negative Pair Consistency:** Unlike shallow models where negative pairs collapsed toward lower distance ranges, ResNet-50 maintains an extended negative distribution shifting toward $d \approx 0.6$–0.8.
- **Batch Size Dynamics:** The reduced batch size ($B = 16$) introduces higher gradient variance during back-propagation, which acts as implicit regularization but also constrains contrastive negative sampling within each optimization step.

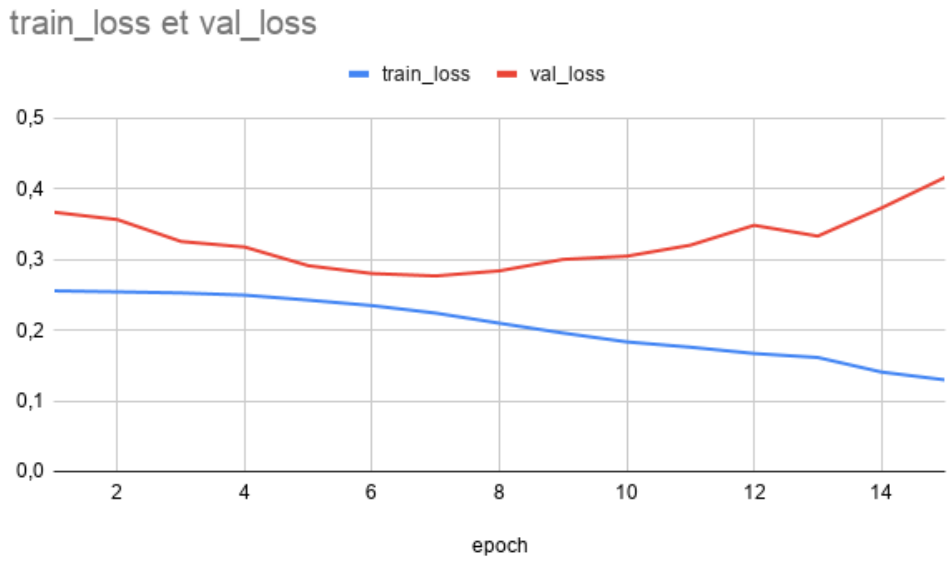
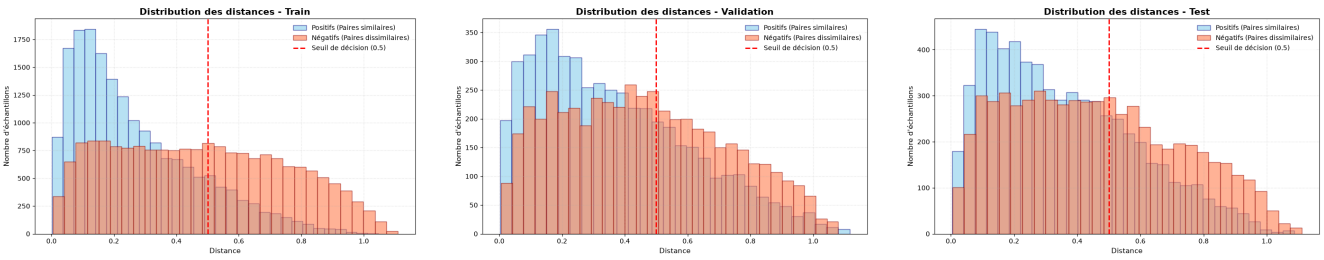


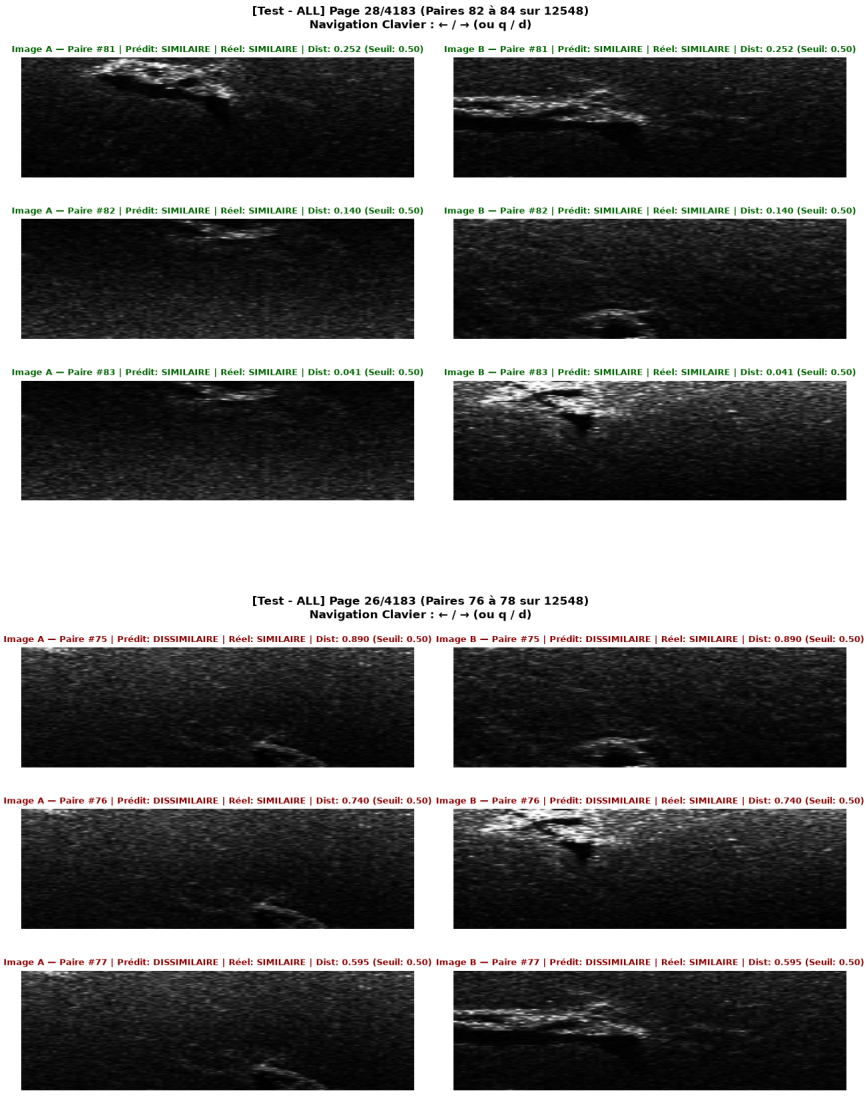
Figure 33: Training and validation loss evolution across epochs for the Siamese ResNet-50 architecture.



(a) Training set (b) Validation set (c) Test set

Figure 34: Pairwise Euclidean distance distributions for the ResNet-50 model across dataset splits.

- **Generalization Horizon:** Although the model remains limited by the widespread acoustic homogeneity of the benthic environment, the increased receptive field and residual connections allow ResNet-50 to extract more discriminative representations without manual feature engineering. Further exploration using self-supervised pre-training or aggressive data regularizations (e.g., Mixup, CutBlur) presents a compelling direction.



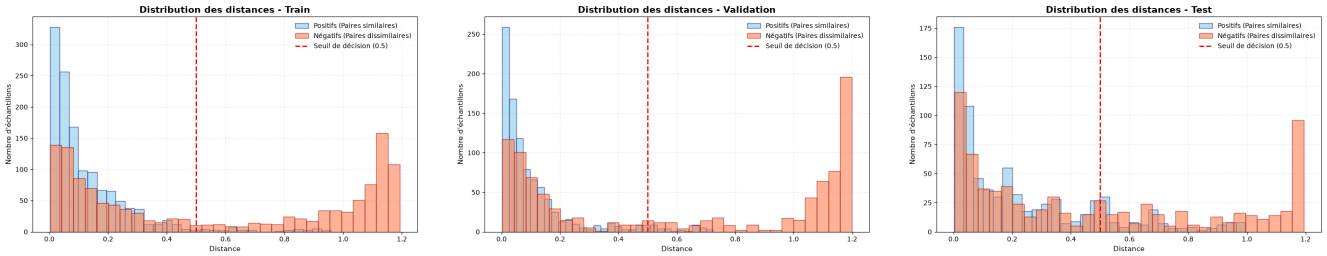
4.0.8 Experiment 16: Evaluation of the ResNet-50 Backbone on HP-Centered Dataset

To establish the upper-bound discriminatory capacity of deeper convolutional representations under favorable acoustic conditions, the Siamese **ResNet-50** architecture was evaluated on the benchmark of distinctive structural regions (`HP-Centered-100`).

Discriminative Capability and Confusion Matrix Dynamics On the `HP-Centered-100` dataset, ResNet-50 achieves the highest overall classification performance among all evaluated models, reaching an accuracy of 74.74% on the validation split and 63.30% on the held-out test split (see Table 3).

A comparative breakdown with the shallow baseline (`S-3CNN-A`, Experiment 7) reveals key behavioral characteristics:

- **True Positive Sensitivity:** The architecture demonstrates a very high recall on positive pairs, achieving 631 True Positives against only 117 False Negatives on the test split. The residual connections allow the encoder to effectively preserve fine-grained acoustic salient structures (e.g., shipwrecks, rocky bedforms) across varying survey passes.



(a) Training set (b) Validation set (c) Test set

Figure 35: Pairwise Euclidean distance distributions for the ResNet-50 architecture trained on the `HP-Centered-100` database across dataset splits.

- **Latent Space Bimodality:** As depicted in Figure 35, the distance distributions exhibit a clearer bimodal separation than on the continuous grid. The impostor (negative) pair distribution shows a pronounced peak centered near $d \approx 0.7$–0.9, confirming that structurally distinct landmarks are mapped further apart in the latent metric space.
- **Persistent Low-Distance False Positives:** Despite these architectural gains, an anomalous concentration of negative pairs persists in the lower distance regime ($d < 0.4$), accounting for 432 False Positives on the test set. This behavior stems from pairs containing homogeneous acoustic textures or similar reverberation patterns: when two distinct patches both exhibit uniform diffuse scattering, the network projects their embeddings closely together, mistaking acoustic uniformity for structural identity.

5 Conclusion

Throughout the various experiments conducted in this study, automatic visual place recognition and drift recalibration for Autonomous Underwater Vehicles (AUVs) using a purely data-driven neural network appear challenging on this dataset. In this real-world acoustic dataset, the vast majority of sonar patches extracted from the uniform spatial grid capture homogeneous seafloor lacking distinctive morphological landmarks. Salient acoustic keypoints and structural landmarks are inherently sparse across realistic seabed surveys. Consequently, a direct end-to-end place recognition approach using the baseline Siamese architectures evaluated here is not directly viable without auxiliary constraints.

Several avenues for improvement can be pursued. First, the effective dataset volume could be expanded. In our current pipeline, we exclusively retain sonar pings acquired concurrently with valid phase bathymetry to preserve local depth profiles, which discards a fraction of acoustic pings. An alternative approach would involve generating a continuous global bathymetric digital elevation model and interpolating the nearest estimated depth for every recorded ping, thereby maximizing data utilization.

Another promising direction lies in transfer learning and synthetic pre-training. Pre-training deep encoders on optical domain datasets or training foundational acoustic models on simulated side-scan sonar data could establish robust low-level filtering and edge-detection priors before fine-tuning on real underwater imagery. High-fidelity acoustic simulators represent an effective strategy to mitigate data scarcity and sensor noise by providing controlled reverberation, speckle characteristics, and varied viewpoints. Exploring realistic sonar rendering frameworks constitutes a valuable next step.

Nevertheless, the evaluated Siamese models successfully identify the presence of salient structural anomalies, as demonstrated by the experiments on the hand-picked benchmark dataset (`HP-Centered-100`). While the network reliably detects structural targets against uniform background reverberation, it struggles to discriminate between different objects under extreme viewpoint variations and acoustic speckle noise—a perceptual challenge that is equally difficult for human operators unfamiliar with raw side-scan echograms.

For practical AUV navigational recalibration, a hybrid framework could be envisioned: rather than performing unconstrained global place recognition, the vehicle could query a prior bathymetric/morphological map, predict expected acoustic landmarks along its trajectory, detect their presence, and compute the spatial offset (shift) to correct the dead-reckoning drift.

6 Bibliography

References

[1] Cyril Chailloux, Jean-Marc Le Caillec, Didier Gueriot, and Benoit Zerr. Intensity-based block matching algorithm for mosaicing sonar images. *IEEE Journal of Oceanic Engineering*, 36(4):627–645, 2011.

[2] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. *arXiv preprint arXiv:2002.05709*, 2020.

[3] EdgeTech. *EdgeTech 6205 Combined Bathymetry and Side Scan Sonar Datasheet*. EdgeTech Marine, West Wareham, MA, USA. Technical specification sheet. Available at: `https://cdn.macartney.com/media/6147/6205-swath-bathymetry-side-scan-sonar.pdf`.

[4] Charles Elkan. Thresholding classifiers for cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, volume 1, pages 625–632, 2001.

[5] Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimensionality reduction by learning an invariant mapping. In *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, volume 2, pages 1735–1742. IEEE, 2006.

[6] Yasmine Khenchaf and Abdelmalek Toumi. Siamese neural network for automatic target recognition using synthetic aperture radar. In *IGARSS 2023 - 2023 IEEE International Geoscience and Remote Sensing Symposium*, pages 7503–7506, 2023.

[7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 25:1097–1105, 2012.

[8] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[9] Subhashree Mishra, Soumya K. De, and Parthasarathi Majumdar. Self-supervised feature representation learning for underwater sonar imagery using contrastive loss. *IEEE Journal of Oceanic Engineering*, 46(3):912–924, 2021.

[10] Evgenia Rusak, Patrik Reizinger, Attila Juhos, Oliver Bringmann, Roland S. Zimmermann, and Wieland Brendel. Infonce: Identifying the gap between theory and practice. *arXiv preprint arXiv:2407.00143*, 2024.

[11] H. Thomas, Christophe Collet, Koffi Yao, and Gilles Burel. Sonar texture classification and rotation invariance. *Traitement du Signal*, 01 2000.

[12] Peter Vandrish, Andrew Vardy, Donald Walker, and Octavia A. Dobre. Side-scan sonar image matching using siamese convolutional neural networks. In *Proceedings of the IEEE/OES Autonomous Underwater Vehicles (AUV)*, pages 1–6. IEEE, 2014.

[13] Xinyu Wang, Sheng Zhang, Zheng Liu, and Peng Chen. A siamese architecture for automatic target recognition and matching in synthetic aperture sonar (sas) imagery. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–14, 2022.

[14] Lotfi A Zadeh. Fuzzy sets. *Information and Control*, 8(3):338–353, 1965.

A Utilisation du code et guide technique

Le projet est développé en Python (testé avec Python 3.10+) en s'appuyant sur les bibliothèques scientifiques `PyTorch`, `NumPy`, `Matplotlib`, `xarray` et `netCDF4`.

Le code source est disponible sur le dépôt Git :

`https://github.com/Kihax/sonar-database-creator`

Pour initialiser le projet, il suffit de cloner le dépôt :

```
git clone https://github.com/Kihax/sonar-database-creator
```

Une fois le dépôt cloné, il est nécessaire de télécharger les données brutes au format NetCDF[1] et de les disposer selon l'arborescence ci-dessous :

```
sonar-database-creator/
 analyse_donnee/              # Scripts d'analyse exploratoire et tracés
 configs_experience/          # Fichiers de configuration (ex. experience5.py)
 create_databases/            # Scripts de génération des bases d'imagettes
    create_db_imagette...    # Échantillonnage sur grille spatiale
    create_db_interest...    # Extraction sur zones d'intérêt
    create_db_similarity...  # Construction des paires de similarité
 database/                    # Bases de données générées finales
 Dataset Metric/              # Fichiers bathymétriques bruts (.nc)
 epochs/                      # Historiques et courbes d'entraînement
 lib/                         # Fonctions utilitaires, parsing et datasets
 model/                       # Sauvegardes des modèles entraînés (.pt / .pth)
 nn_model/                    # Fichiers pour lancer l'entrainement ou analyser les performances du modèle
 refactored_data/             # Données tampons pré-calculées
 create_dbs_tempon.py         # Script de création de la base tampon
```

> **Remarque importante :** Le dossier `refactored_data/` doit impérativement être créé à la racine avant d'exécuter le script `create_dbs_tempon.py`, sans quoi Python retournera une erreur système (`FileNotFoundError`).

A.1 Création des sous-bases de données tampons (`create_dbs_tempon.py`)

Le temps de traitement pour accéder aux informations de bathymétrie directement dans les fichiers bruts est particulièrement long. Nous créons donc des bases de données tampons pour éviter de recalculer la bathymétrie à chaque expérimentation.

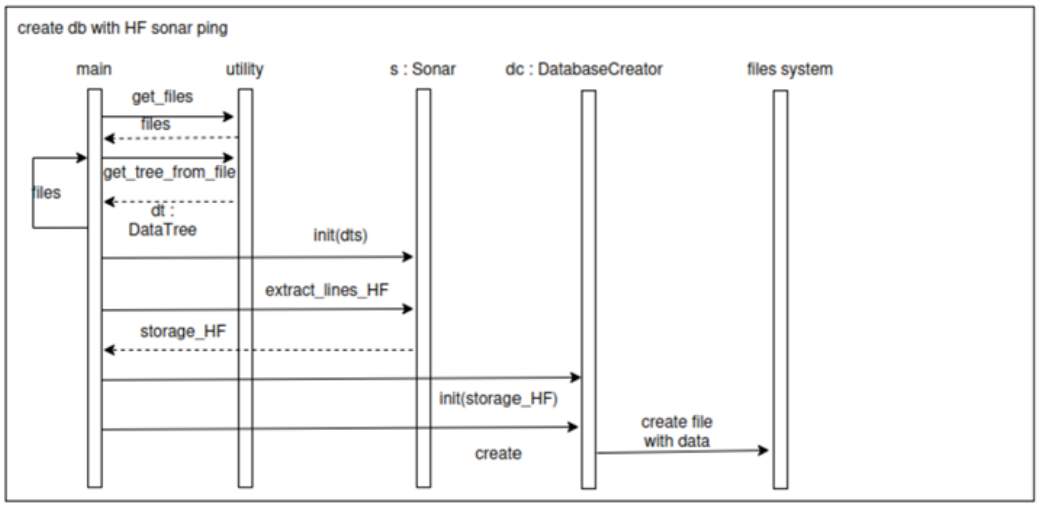


Figure 36: Schéma du processus de création de la base de données tampon.

[1] Fichiers téléchargeables à l'adresse : `https://drive.google.com/file/d/1EmbqSpKM22kdebAd3c_iXQ65SHLZ6Kig/view?usp=sharing` envoyer un email à kerlau.killian@imt-atlantique.net et didier.gueriot@imt-atlantique.fr pour y avoir accès

Lors de cette étape, seules les grandeurs nécessaires pour la suite sont conservées : position corrigée du navire, profondeur, ping sonar brut, pas d'échantillonnage temporel Δt entre chaque sample, etc.

Le chemin spécifié dans la fonction `get_files` correspond au dossier où sont stockés les pings sonars bruts dans les fichiers `.nc` (`Dataset Metric/`). Pour ajouter ou retirer des informations lors de l'écriture du fichier `.nc`, il suffit de modifier la classe chargée de générer ce fichier dans la bibliothèque `lib/`. De même, pour adapter l'extraction des pings sur un fichier `.nc` modifié, il conviendra d'ajuster les classes correspondantes dans `lib/` et d'exécuter `create_dbs_tempon.py` en indiquant le bon chemin.

A.2 Création des bases de données d'imagettes (`create_databases/`)

La création des bases de données d'imagettes s'appuie sur les bases de données tampons générées précédemment afin d'extraire les imagettes et de sélectionner les métadonnées pertinentes selon le cas d'usage.

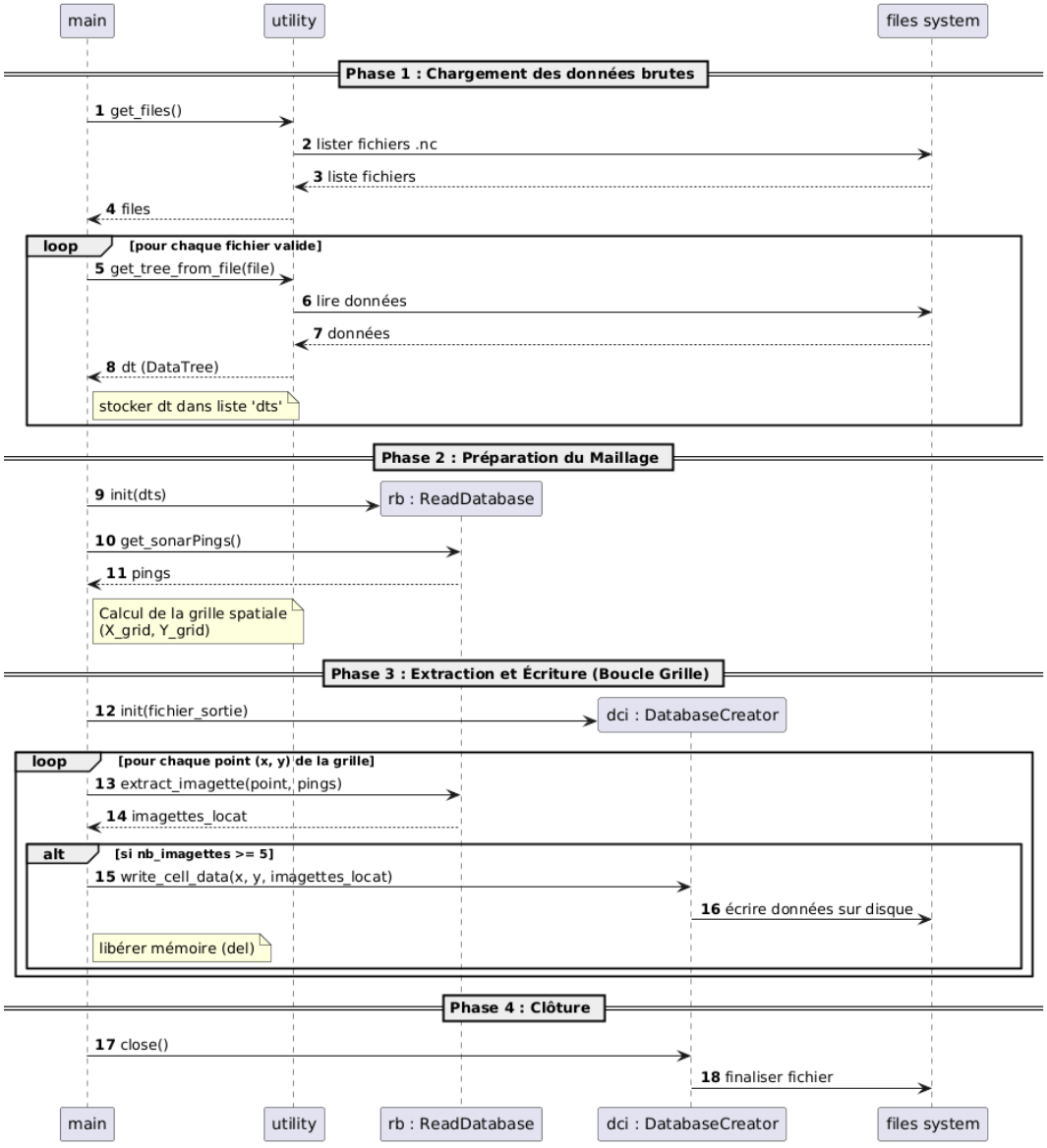


Figure 37: Génération de la grille de navigation et extraction spatiale des imagettes.

Le script projette une grille spatiale le long des positions parcourues par le navire, puis applique une fonction d'extraction (`extract_imagette`) pour chaque position afin de construire la base de données de manière parallélisée.

A.3 Dataset PyTorch et pipeline d'entraînement

A.3.1 Formatage et normalisation

Pour modifier rapidement les données exploitées sur une base déjà générée, les ajustements s'effectuent dans le module de dataset PyTorch de `lib/`. Ce dernier prend en charge :

- La normalisation des valeurs acoustiques et des métadonnées ;
- Le formatage de l'entrée du réseau (soit sous forme d'image monocanal avec métadonnées scalaires associées, soit sous forme d'image multicanale où le cap est transformé en matrice 2D répliquée) ;
- Le choix du nombre de canaux d'entrée (*grayscale* : 1, multimodal : ≥ 2).

A.3.2 Génération des paires

La constitution du dataset de paires est réalisée lors de l'initialisation de l'entraînement :

- Les couples positifs sont formés en appariant le nombre maximal de paires issues d'une même zone géographique sous différents passages ;
- Les couples négatifs sont générés en tirant une seconde imagette de façon aléatoire ;
- Une graine pseudo-aléatoire (`seed = 42`) est fixée pour garantir la reproductibilité intégrale des découpages de données.

A.3.3 Exécution de l'entraînement

L'entraînement est lancé via le script principal :

```
python code/train_nn.py
```

Une ligne peut être changée pour changer la configuration Le script initialise le dataset, découpe les zones géographiques en sous-ensembles d'apprentissage et de validation, instancie l'architecture siamoise sélectionnée (ex. *S-3CNN*) et applique la fonction de perte retenue (*Contrastive Loss* de Hadsell ou *InfoNCE*).

A.4 Entraînement avec différentes métadonnées et création de nouveaux modèles

A.4.1 Modification des métadonnées d'entraînement

L'entraînement d'un modèle existant avec différentes combinaisons de métadonnées s'effectue directement via les fichiers de configuration du dossier `configs_experience/` (ex. `experience5.py`), sans modifier le cœur de la boucle d'apprentissage.

A.4.2 Intégration d'une nouvelle architecture neuronale

Pour concevoir un nouveau modèle de réseau siamois :

1. Écrire la nouvelle classe PyTorch dans le dossier `model/`, en s'inspirant des architectures existantes (`SiameseSonarNetwork.p`
   Le modèle doit intégrer la possibilité de fusionner les métadonnées scalaires ou d'adapter ses canaux d'entrée matriciels.
2. Déclarer et importer ce nouveau modèle dans le fichier (`lib/get_model.py`) afin de lier le nom de la configuration à l'instanciation de la classe.
3. Mettre à jour le fichier de configuration associé dans `configs_experience/` pour lancer l'expérience.

B Use of Generative AI

Generative AI tools (Large Language Models) were utilized during the preparation of this manuscript strictly for language refinement, academic English translation, and assistance with LaTeX formatting and structural code organization. All conceptual formulations, geometric and acoustic modeling, algorithmic implementations, experimental designs, and data analyses were entirely conceived, conducted, and critically verified by the author.